

ReactJS 라이브러리

FrontEnd



<https://react.dev/>



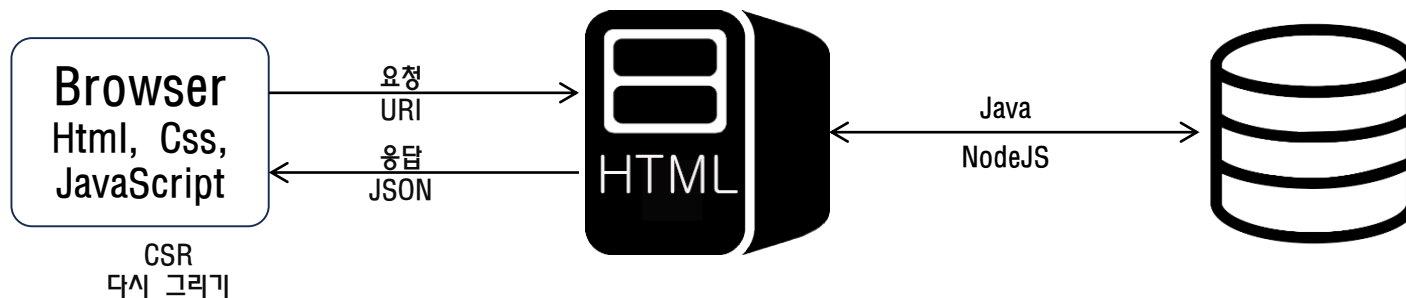
≡ Client-Side Rendering

웹 페이지의 콘텐츠를 보여주는 동작을 브라우저(클라이언트)에서 JavaScript를 사용하여 처리(렌더링)하는 방식입니다.

Client-Side Rendering(CSR)은 서버에서 기본적으로 HTML이나 JSON 형식의 데이터를 전달하고, JavaScript가 실행되면서 브라우저에서 화면이 완성됩니다.

데이터 제공은 주로 REST Api 형식으로 개발이 됩니다.

웹 페이지의 주요 콘텐츠는 서버에서 생성되는 것이 아니라, 브라우저에서 JavaScript를 통해 동적으로 구성됩니다.



CSR의 동작 과정

사용자가 웹사이트에 데이터를 요청(예: `example.com`)합니다.

서버는 기본적인 HTML 파일 혹은 JSON을 반환합니다.

HTML 파일에 포함된 JavaScript(주로 React, Vue, Angular 등)가 실행이 됩니다.

JavaScript가 API 요청을 보내 필요한 데이터를 가져옵니다.

데이터를 기반으로 동적으로 UI를 생성하여 사용자에게 표시해 줍니다.

≡ CSR의 장단점

CSR의 장단점에 대하여 살펴 보겠습니다.

CSR의 장점

항목	설명
빠른 페이지 전환	한 번 로드된 후에는 새로운 페이지를 불러올 때 서버 요청 없이 즉시 전환 가능 (SPA, Single Page Application).
동적 웹 애플리케이션에 적합	AJAX, Fetch API, GraphQL 등을 활용하여 실시간 데이터 업데이트 가능. 예: React, Vue, Angular로 개발된 웹 앱 (Twitter, Facebook, Gmail 등).
서버 부하 감소	서버는 최소한의 HTML과 JavaScript만 전달하므로 부하가 적고 확장성(Scalability)이 높음.
모바일 앱과 유사한 UX 제공	CSR 방식의 앱은 브라우저에서 빠르게 렌더링되어 앱과 유사한 경험(PWA, Progressive Web App) 제공.

CSR의 단점

항목	설명
초기 로딩 속도 느림 (First Contentful Paint 지연)	JavaScript 파일을 다운받아 실행해야 하기 때문에, 첫 화면을 표시하는 속도가 느릴 수 있음. 네트워크 상태가 좋지 않거나, 저사양 기기에서는 렌더링이 오래 걸릴 수 있음.
검색 엔진 최적화(SEO) 문제	HTML이 처음에는 비어 있는 상태로 제공되므로, Google 등 검색 엔진 크롤러가 콘텐츠를 인식하기 어려움. 해결 방법: SSR(Server-Side Rendering) 또는 Hydration 방식 사용.
JavaScript 의존도 높음	클라이언트(사용자) 측에서 JavaScript가 비활성화된 경우, 웹사이트가 제대로 작동하지 않을 수 있음.

≡ Frontend 개요

사용자가 직접 보고, 상호 작용하는 웹사이트나 애플리케이션의 UI(User Interface) 부분을 개발하는 영역입니다.

사용자는 프론트엔드를 통해 데이터를 입력하고, 결과를 확인하며, 서버와 상호 작용할 수 있습니다.

Frontend의 주요 역할

프론트엔드는 단순한 디자인이 아니라, 사용자 경험을 최적화하고, 동적인 데이터를 처리하는 역할을 합니다.

주요 역할은 다음과 같습니다.

항목	설명
UI(User Interface) 구현	HTML, CSS, JavaScript를 사용하여 웹사이트의 구조, 디자인, 레이아웃을 구성. 반응형 웹(Responsive Web)을 적용하여 다양한 기기(PC, 모바일)에서 최적화된 화면 제공.
UX(User Experience) 개선	사용자가 직관적으로 웹사이트를 사용할 수 있도록 애니메이션, 전환 효과, 인터랙션(클릭, 스크롤 등) 적용. 성능 최적화를 통해 빠른 로딩 속도를 제공.
데이터 처리 및 서버와 통신	JavaScript를 활용한 동적 웹사이트 개발(ex: 버튼 클릭 시 UI 변경). AJAX, Fetch API, Axios 등을 사용하여 서버에서 데이터를 가져와 화면에 반영. REST API, GraphQL 등을 통해 백엔드와 연동.
상태 관리 및 이벤트 처리	전역 상태 관리 라이브러리 사용(Redux, Recoil, Zustand 등). 컴포넌트 간 데이터 전달 및 공유. 이벤트 리스너(event listener) 활용(ex: onClick, onChange).

≡ Frontend 개발 핵심 기술

프론트엔드 개발에는 다음과 같은 다양한 기술들이 사용될 수 있습니다.

개발 핵심 기술

항목	설명
기본 웹 기술	HTML(HyperText Markup Language) → 웹 페이지의 뼈대를 정의. CSS(Cascading Style Sheets) → 색상, 레이아웃, 애니메이션 등 스타일 지정. JavaScript(JS) → 사용자와의 상호 작용을 가능하게 하는 프로그래밍 언어.
CSS 프레임워크 & 전처리기	Bootstrap: 빠른 UI 개발을 위한 프레임워크. Tailwind CSS: 유틸리티 기반의 스타일링 도구. SASS/SCSS, LESS: CSS 전처리기(Preprocessor), 코드 재사용 및 유지보수성 향상.
JavaScript 프레임워크 & 라이브러리	React.js(가장 인기 있는 라이브러리) Vue.js(간결하고 쉬운 구조) Angular(구글에서 개발, 대규모 프로젝트에 적합)
상태 관리 라이브러리	Redux: 전역 상태 관리(React와 자주 사용됨). Recoil, Zustand, MobX: React의 상태 관리 대체 라이브러리.
백엔드와의 데이터 통신	REST API / GraphQL 사용. Axios, Fetch API를 활용하여 백엔드와 데이터 주고 받기.
빌드 도구 & 패키지 관리자	Webpack, Vite, Parcel: 코드 번들링 및 최적화. NPM(Node Package Manager), Yarn, PNPM: 라이브러리 및 패키지 관리.

≡ 프론트엔드 개발자의 역할

프론트엔드 개발자로서의 역할에 대하여 살펴 봅니다.

프론트엔드 개발자의 역할

UI/UX 설계 및 디자인 반영

웹사이트의 성능 최적화 및 반응형 개발

API 연동 및 서버와의 데이터 통신 처리

웹 접근성 및 SEO(Search Engine Optimization) 최적화

최신 웹 기술 및 프레임워크 활용

ReactJS 라이브러리

React의 개요와 특징

The React logo, consisting of the word "React" in a bold, blue, sans-serif font, centered within a dark blue rectangular background.

Edit src/App.js and save to reload.

[Learn React](#)

React의 개요와 특징

React 소개

컴포넌트 기반의 웹 애플리케이션을 만들 수 있도록 설계되어 있는 JavaScript 기반 UI 라이브러리입니다.

React는 Facebook(현재 Meta)에서 개발한 클라이언트 측 자바스크립트 라이브러리입니다.



항목	설명
클라이언트(React)	사용자 인터페이스를 구축하고, 서버에 API 요청을 보냅니다.
서버(Apache Tomcat 등)	API 요청을 받아서 데이터 베이스와 통신합니다.
데이터 베이스	데이터를 저장하고, 요청에 따라 데이터를 제공합니다.

☰ React의 개요와 특징

☰ React 소개

FaceBook에서 만든 사용자 인터페이스(UI)를 위한 자바스크립트 라이브러리입니다.

리액트 소개

UI를 생성하기 위한 자바 스크립트 라이브러리입니다.

단일 페이지 애플리케이션을 만들기(build) 위하여 사용됩니다.

개발자에게 재사용 가능한 UI component를 제공해 줍니다.

React JS는 프레임워크가 아닌 라이브러리입니다.

- 프레임워크는 틀에서 벗어나기가 어렵습니다.
- 라이브러리는 필요한 곳에서 사용하면 됩니다.

React 라이브러리의 핵심은 Virtual DOM을 사용하는 것입니다.

- 실제 DOM을 처리하는 것은 성능도 느리고 비효율적입니다.
- 처리해야 할 데이터가 많아지면 관리하기도 힘듭니다.

React의 개요와 특징

React 소개

FaceBook에서 만든 사용자 인터페이스(UI)를 위한 자바스크립트 라이브러리입니다.

리액트의 특징

항목	설명
컴포넌트 기반 개발 (Component-Based)	UI를 여러 개의 독립된 컴포넌트(Component) 로 나누어 개발. 코드의 재사용성이 높아 유지 보수와 확장성이 뛰어남.
Virtual DOM(가상 DOM) 사용	React는 Virtual DOM을 사용하여 변경된 부분만 업데이트(효율적인 렌더링). 기존 DOM을 직접 수정하는 방식보다 성능이 뛰어남.
단방향 데이터 흐름 (One-way Data Binding)	데이터는 부모 → 자식 컴포넌트로만 전달되며, 예측 가능한 흐름을 유지. props를 이용하여 데이터를 전달.
선언형 프로그래밍 (Declarative Programming)	React는 UI를 선언적으로 정의하여 코드 가독성이 높음. HTML처럼 JSX 문법을 사용하여 간결한 코드 작성 가능.

☰ React의 개요와 특징

☰ React의 장점과 단점

React가 가질 수 있는 장점과 단점은 대략 다음과 같습니다.

장점과 단점

항목	설명
Virtual DOM을 사용합니다.	작고, 전체 웹 개발 중 단 하나의 부분만 담당
배우기가 쉽습니다.	단지 View를 위한 라이브러리입니다.
컴포넌트를 사용하여 가독성을 높여 줍니다.	데이터 모델링, 라우팅, Ajax 등은 사용 불가능합니다.
뛰어난 Garbage Collection 관리 기능	IE8 이하 버전은 지원하지 않습니다.
매우 간편한 UI 수정 및 재사용	
FaceBook이 지속적으로 지원해 줍니다.	
다른 프레임워크나 라이브러리와 혼용 가능	

React의 개요와 특징

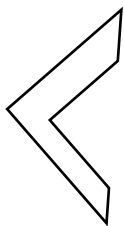
React의 특징

React는 다음과 같은 특징들을 가지고 있습니다.

리액트 특징

새로운 ECMAScript(에크마 스크립트) 문법(ES6라고 부름)을 사용할 수 있습니다.
클래스형(객체 지향)과 함수형 자바 스크립트 2가지가 존재합니다.
클래스형(객체 지향) 보다는 함수형 자바 스크립트를 더 선호합니다.
컴포넌트 기반의 라이브러리입니다.

클래스
타입



함수
타입



컴포넌트 기반



☰ React의 개요와 특징

☰ React의 주요 개념

React는 다음과 같은 주요 개념들을 가지고 있습니다.

React의 주요 개념

개념	설명	예시
JSX(JavaScript XML)	JavaScript 안에서 HTML과 유사한 문법을 사용하여 UI를 쉽게 작성할 수 있음.	<code>const element = <h1>Hello, React!</h1>;</code>
컴포넌트 (Component)	UI를 독립적인 컴포넌트 단위로 쪼개어 개발 가능.	<code>function Welcome(props) { return <h1>Hello, {props.name}!</h1>; }</code>
Props(속성)	부모 컴포넌트에서 자식 컴포넌트로 데이터를 전달할 때 사용.	<code>function Greeting(props) { return <h1>안녕하세요, {props.name}님!</h1>; } <Greeting name="철수" />;</code>

☰ React의 개요와 특징

☰ React의 주요 개념

React는 다음과 같은 주요 개념들을 가지고 있습니다.

React의 주요 개념

개념	설명	예시
State(상태)	컴포넌트 내부에서 데이터를 관리하는 변수.	<pre>import { useState } from "react"; function Counter() { const [count, setCount] = useState(0); return(<div> <p>현재 카운트: {count}</p> <button onClick={() => setCount(count + 1)}>+</button> </div>); }</pre>
이벤트 핸들링	React에서 이벤트를 처리하는 방식.	<pre>function ClickButton() { function handleClick() { alert("버튼 클릭됨!"); } return <button onClick={handleClick}>클릭</button>; }</pre>

☰ React의 개요와 특징

☰ React 기초

다음 항목들은 React에서 자주 반복되는 용어들입니다.

기본 용어	설명
ES6	2015년에 업그레이드된 공식 자바 스크립트 문법입니다.
Redux(리덕스)	페이스 북의 데이터 아키텍처 라이브러리입니다. 애플리케이션의 상태(state)를 관리해 줍니다.
WebPack	모듈 번들러라고 부르며, 프로젝트에서 분리된 여러 파일을 빌드하는 데 사용합니다.
Express	노드 JS 환경에서 웹 서버를 쉽게 연동하게 해주는 FrameWork입니다.
mongoose	노드 JS 환경에서 MongoDB를 쉽게 연동하게 해주는 FrameWork입니다.
Babel	최신 자바 스크립트 문법을 이해하지 못하는 브라우저를 위한 JSX 컴파일 Tool입니다.
ESLint	JavaScript 코드에서 일관성, 가독성 및 에러를 검사하는 린트 도구(Linting Tool) 중 하나입니다. 즉, 개발자가 작성한 JavaScript 코드를 분석하여 코드 스타일, 잠재적인 오류 및 버그를 검사하고 수정하는 데 도움을 주는 도구입니다.

React의 개요와 특징

리액트 관련 프로그램 설치

리액트 실습을 위해서는 다음과 같은 프로그램들이 설치/셋팅이 되어야 합니다.

설치 항목

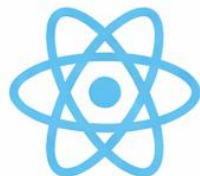
React를 실습하기 위해서는 다음과 같은 프로그램을 설치해야 합니다.

항목	설명
Node.js 및 npm (Node Package Manager)	리액트 애플리케이션을 실행하고 관리하기 위해 필요합니다. Node.js를 설치하면 npm도 함께 설치됩니다.
Code Editor (코드 편집기)	Visual Studio Code, Sublime Text, Atom 등의 인기 있는 코드 편집기를 사용할 수 있습니다. 원하는 코드 편집기를 다운로드하여 설치합니다.

Node JS 설치



vite로 설치하기



React

Build the UI



Vite

Fast Run & Build



Node.js

Run Everything

React의 개요와 특징

vite로 설치하기

Vite로 React 프로젝트를 생성하는 방법에 대하여 살펴 봅니다.

설치 하기

원하는 폴더에서 다음과 같이 실행합니다.

c:\ 드라이브 혹은 d:\ 드라이브에서 시작합니다.

```
npm create vite@latest 엔터
```

Need to install the following packages:

create-vite@8.3.0

Ok to proceed? (y) **y**

```
> npx
```

```
> create-vite
```

```
|
```

```
o Project name:
```

```
| react_coffee
```

```
|
```

```
o Select a framework:
```

```
| React
```

```
|
```

```
o Select a variant:
```

```
| TypeScript + React Compiler
```

```
|
```

```
o Install with npm and start now?
```

```
| Yes
```

```
|
```

```
o Scaffolding project in D:\react_coffee...
```

```
|
```

```
o Installing dependencies with npm...
```

```
added 179 packages, and audited 180 packages in 3m
```

React의 개요와 특징

vite로 설치하기

Vite로 React 프로젝트를 생성하는 방법에 대하여 살펴 봅니다.

설치 하기

```
46 packages are looking for funding
  run `npm fund` for details
```

```
found 0 vulnerabilities
```

```
|
```

```
o Starting dev server...
```

```
> react_coffee@0.0.0 dev
```

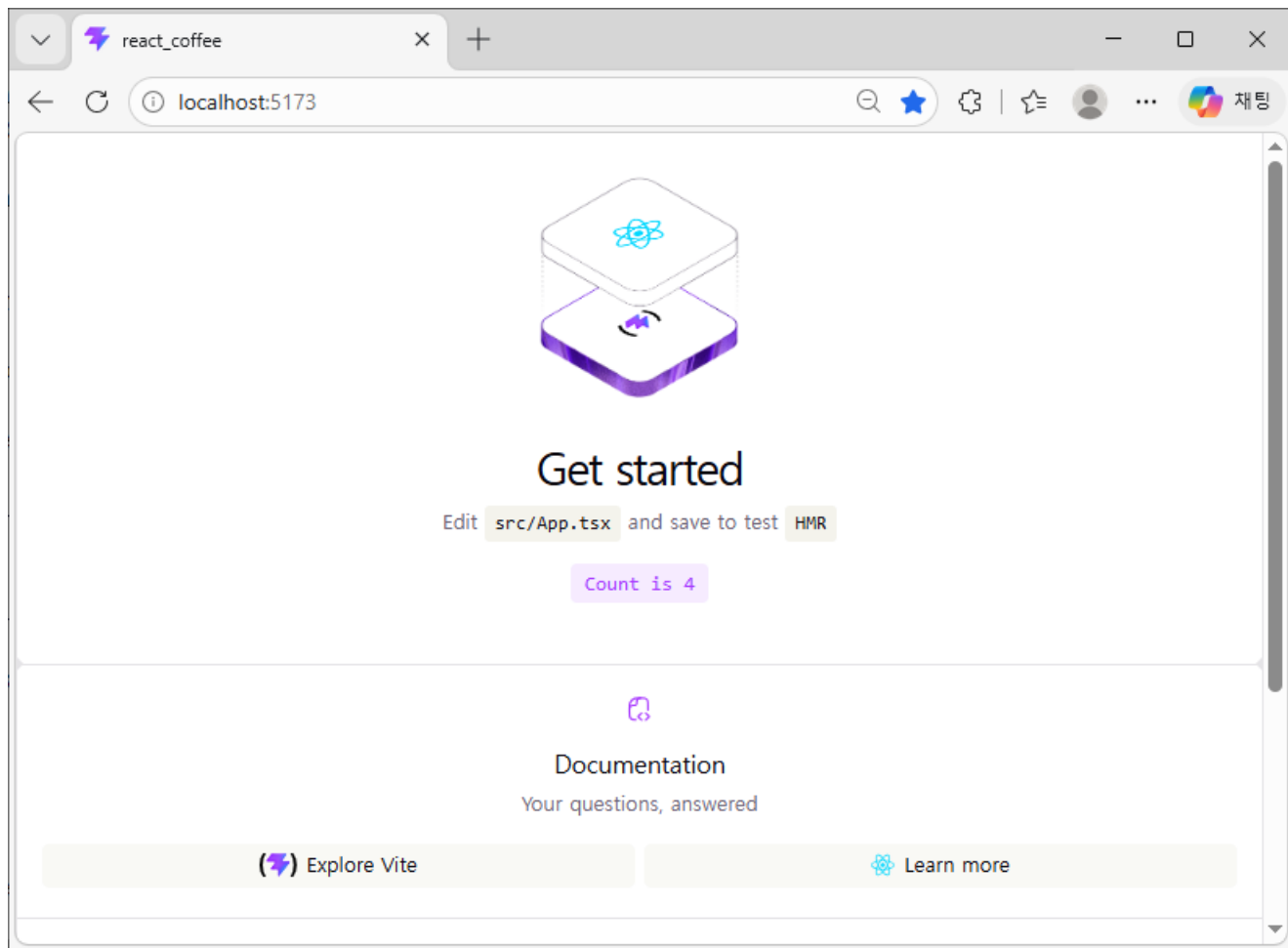
```
> vite
```

```
VITE v7.3.1 ready in 6769 ms
```

```
→ Local: http://localhost:5173/
```

```
→ Network: use --host to expose
```

```
→ press h + enter to show help
```



☰ React의 개요와 특징

☰ 리액트 관련 프로그램 설치

리액트 실습을 위해서는 다음과 같은 프로그램들이 설치/셋팅이 되어야 합니다.

설치 항목

React를 실습하기 위해서는 다음과 같은 프로그램을 설치해야 합니다.

항목	설명
Node.js 및 npm (Node Package Manager)	리액트 애플리케이션을 실행하고 관리하기 위해 필요합니다. Node.js를 설치하면 npm도 함께 설치됩니다.
Code Editor (코드 편집기)	Visual Studio Code, Sublime Text, Atom 등의 인기 있는 코드 편집기를 사용할 수 있습니다. 원하는 코드 편집기를 다운로드하여 설치합니다.

Node JS 설치



create react app 설치

React의 개요와 특징

create-react-app

React 애플리케이션을 쉽게 만들 수 있도록 Facebook(메타)이 개발한 공식 도구입니다.

create react app

복잡한 설정 없이 React Web Application을 쉽게 설정하고 개발할 수 있도록 도와 주는 도구입니다.

필요한 구성과 설정을 자동으로 처리하여 초보자부터 전문가까지 다양한 개발자들이 React 앱을 효율적으로 구축할 수 있도록 설계되었습니다.

간단히 줄여서 CRA라고 부르기도 합니다.

주요 특징

항목	설명
간편한 설정	모든 중요한 설정이 자동으로 처리되며, 기본적인 파일 구조와 설정이 제공됩니다. 복잡한 빌드 도구를 신경 쓰지 않고도 바로 React 개발을 시작할 수 있습니다.
개발 서버 내장	프로젝트를 생성하면 npm start 명령어로 간단히 개발 서버를 실행할 수 있습니다. 이 서버는 자동으로 코드 변경을 감지하고, 브라우저를 새로고침하지 않아도 즉시 변경 사항이 반영됩니다 (hot-reloading 기능).
최적화된 빌드	npm run build 명령어를 사용하면 애플리케이션이 최적화된 프로덕션 빌드로 패키징됩니다. 이 빌드는 애플리케이션을 배포할 때 사용할 수 있도록 작은 용량과 빠른 성능을 제공하기 위해 최적화됩니다.
JSX 및 최신 JavaScript 지원	React의 JSX 문법을 기본으로 지원하고, 최신 JavaScript 표준을 자동으로 처리하여 브라우저 호환성을 제공합니다.
테스트 환경 제공	Jest를 기반으로 하는 기본 테스트 환경이 제공됩니다. npm test 명령어로 테스트를 실행할 수 있습니다.
ESLint와 Prettier 통합	코드의 일관성을 유지하고 오류를 줄이기 위해 ESLint와 Prettier 같은 도구가 기본적으로 설정되어 있습니다.

☰ React의 개요와 특징

☰ create-react-app

전역 설치와 로컬 설치에 대하여 살펴 봅니다.

전역 설치

컴퓨터의 전체 환경에 설치하여 모든 프로젝트에서 사용할 수 있게 하는 방법을 의미합니다.

서로 다른 프로젝트를 매번 생성할 때 마다 별도의 설치 과정 없이 빠르게 새로운 React 앱을 만들 수 있습니다.

```
npm install -g create-react-app  
create-react-app reactapp
```

지역 설치

npx 명령어를 통하여 한 번만 다운로드하여 바로 React 프로젝트를 생성할 수 있습니다.

이 방식은 최신 버전의 create-react-app을 사용할 수 있는 장점이 있습니다.

```
cd \  
  
npx create-react-app reactapp
```

ReactJS 라이브러리

React를 위한 자바 스크립트

The word "React" in a bold, blue, sans-serif font, centered within a dark blue rectangular background.

Edit src/App.js and save to reload.

[Learn React](#)

☰ React를 위한 자바 스크립트

☰ JavaScript Versions

자바 스크립트에 대하여 간단히 살펴 보도록 하겠습니다.

JavaScript

Brendan Eich에 의하여 1995년도에 개발이 되었고, 1997년에 ECMA(에크마)의 표준이 되었습니다.

ECMAScript is the official name of the language.

ECMAScript versions have been abbreviated to ES1, ES2, ES3, ES5, and ES6.

Since 2016, versions are named by year (ECMAScript 2016, 2017, 2018, 2019, 2020).

ECMAScript

European Computer Manufacturers Association의 줄임말입니다.

ECMA는 1961년에 설립된 국제적인 정보 통신 표준화 기구로, 컴퓨터 시스템과 소프트웨어 기술에 대한 표준을 개발하는 역할을 합니다.

ECMA 인터내셔널은 정보와 통신 시스템에 대한 표준을 제정하는 국제적 표준화 기구입니다.

ECMA 인터내셔널에서 만든 ECMAScript(=ES)의 6번째 개정판 문서에 있는 표준 스펙입니다.

☰ React를 위한 자바 스크립트

☰ Javascript ES6

ES6 또는 ECMAScript 6라고 부르는 ECMAScript 2015는 JavaScript의 두 번째 major revision입니다.

New Features in ES6

The let keyword

The const keyword

Arrow Functions

The {a,b} = Operator

The [a,b] = Operator

The ... Operator

For/of

Map Objects

Set Objects

Classes

Promises

Symbol

Default Parameters

Function Rest Parameter

String.includes()

String.startsWith()

String.endsWith()

Array entries()

Array.from()

Array keys()

Array find()

Array findIndex()

Math.trunc

Math.sign

Math.cbrt

Math.log2

Math.log10

Number.EPSILON

Number.MIN_SAFE_INTEGER

Number.MAX_SAFE_INTEGER

Number.isInteger()

Number.isSafeInteger()

New Global Methods

JavaScript Modules

☰ React를 위한 자바 스크립트

☰ ES 버전 정리

자바스크립트의 ES5, ES6 같은 이름은 사실 ECMAScript(에크마스크립트) 표준의 버전을 의미합니다.

버전별 특징

버전	공식 명칭	발표 연도	주요 특징
ES3	ECMAScript 3	1999	try/catch, do-while, 정규표현식 지원
ES5	ECMAScript 5	2009	"use strict", JSON 지원, Array 메서드(forEach, map, filter 등), Object 메서드
ES6 (ES2015)	ECMAScript 2015	2015	let/const, 화살표 함수, 클래스, 모듈(import/export), Promise, 템플릿 리터럴, 디스트럭처링
ES7 (ES2016)	ECMAScript 2016	2016	Array.prototype.includes, 지수 연산자(**)
ES8 (ES2017)	ECMAScript 2017	2017	async/await, Object 메서드(Object.entries, Object.values)
ES9 (ES2018)	ECMAScript 2018	2018	Rest/Spread 개선, 비동기 반복(for await...of), Promise.finally
ES10 (ES2019)	ECMAScript 2019	2019	Array.flat, Array.flatMap, Object.fromEntries, trimStart/trimEnd
ES11 (ES2020)	ECMAScript 2020	2020	BigInt, nullish coalescing ??, optional chaining ?., Promise.allSettled
ES12 (ES2021)	ECMAScript 2021	2021	String.replaceAll, 논리 할당 연산자(`
ES13 (ES2022)	ECMAScript 2022	2022	Top-level await, Array.at, Class 필드, Private 메서드
ES14 (ES2023)	ECMAScript 2023	2023	Array.findLast, findLastIndex, Symbol 특성 개선
ES15 (ES2024)	ECMAScript 2024	2024	정식 표준화 완료 예정, 다양한 Stage 4 제안 포함

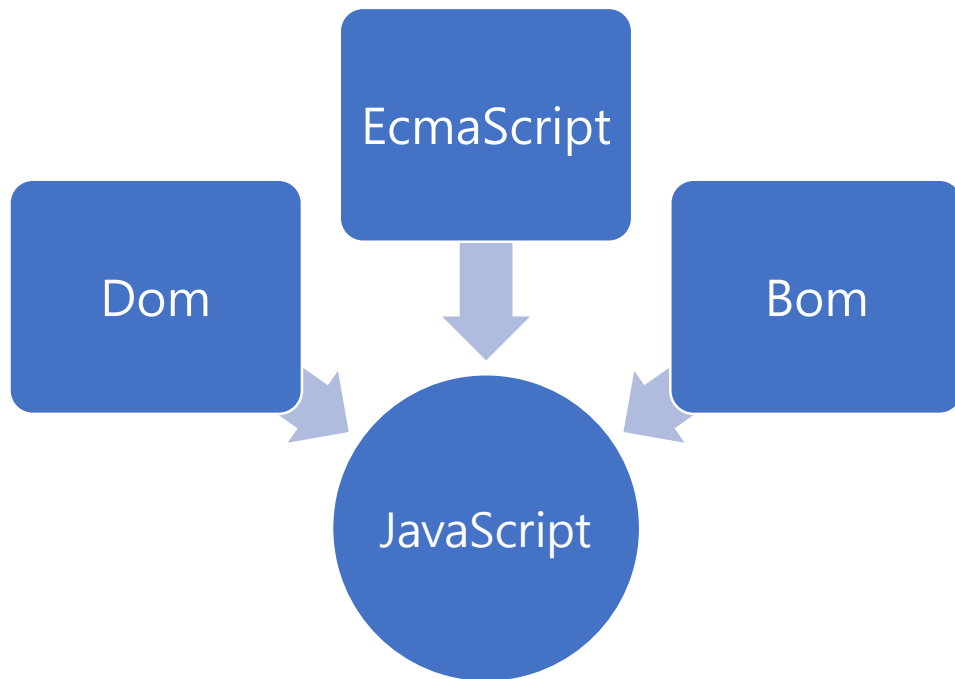
☰ React를 위한 자바 스크립트

☰ React를 위한 자바 스크립트

React와 관련성이 있는 자바 스크립트에 대하여 살펴 보도록 하겠습니다.

JavaScript

JavaScript는 1995년 넷스케이프(Netscape) 브라우저에서 동적인 요소를 구현하기 위해서 개발되었습니다.
JavaScript는 모든 브라우저에서 잘 동작하기 위하여 만든 스크립트 표준입니다.



☰ React를 위한 자바 스크립트

☰ 주석(comment)

JavaScript 주석과 비슷하지만, JSX 문법 안/밖에서 쓰는 방법이 조금 다릅니다.

JSX 밖

```
// 일반 JavaScript 코드 부분에서는 평소처럼
// 기호 또는 /* ... */를 그대로 씁니다.
// 한 줄 주석
const message = "Hello";

/*
여러 줄 주석
여러 줄 모두 적용됨
*/
const name = "React";
```

JSX 안

JSX는 JavaScript 표현식이기 때문에, 주석도 {} 안에 작성해야 합니다.
이때는 반드시 /* ... */ 형식을 써야 해요.

```
function App() {
  return (
    <div>
      /* 이것은 JSX 안에서 쓰는 주석입니다 */
      <h1>Hello React</h1>

      /*
      여러 줄 주석도 가능합니다.
      JSX 안에서는 꼭 중괄호 + 슬래시 별표를 같이 써야 해요.
      */
    </div>
  );
}
```

☰ React를 위한 자바 스크립트

☰ 주석(comment)

JavaScript 주석과 비슷하지만, JSX 문법 안/밖에서 쓰는 방법이 조금 다릅니다.

태그 옆/안

JSX 태그와 같은 줄에도 `{/* ... */}` 형식으로 주석을 넣을 수 있습니다.

```
<div>
  <h1>Hello</h1> {/* 제목 */}
  <p>World</p>    {/* 설명 */}
</div>
```

전체 컴포넌트/엘리먼트

특정 태그 블록 전체를 잠시 숨기고 싶을 때도 자주 씁니다.

```
function App() {
  return (
    <div>
      {/*
        <h1>Hello</h1>
        <p>This is hidden</p>
      */}
    </div>
  );
}
```

☰ React를 위한 자바 스크립트

☰ `var`, `let`, `const`의 차이

자바 스크립트에서 기존의 var 키워드 이외에 let, const 키워드가 추가 되었습니다.

`Scope(스코프)를 설명하기 전에 var, let, const의 차이를 우선 살펴 보도록 합니다.

스코프 유형

키워드	스코프	재선언 가능 여부	재할당 가능 여부	호이스팅
var	함수	가능	가능	가능(초기화 X, undefined)
let	블록	불가능	가능	가능(초기화 X, TDZ 발생)
const	블록	불가능	불가능	가능(초기화 X, TDZ 발생)

TDZ (Temporal Dead Zone): let과 const는 선언 전에 접근할 경우 ReferenceError가 발생하는 영역입니다.

☰ React를 위한 자바 스크립트

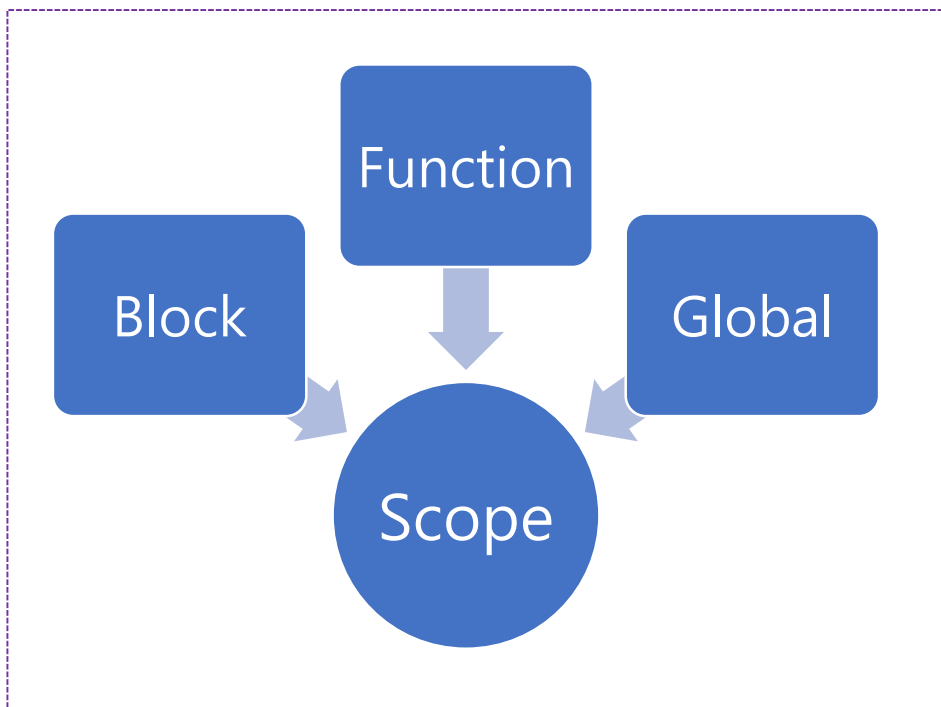
☰ 'Scope

변수, 함수, 객체 등이 접근할 수 있는 범위를 지정해주는 개념입니다.

Scope(스코프)는, 코드에서 특정 변수나 함수가 어디에서 접근 가능하고 사용할 수 있는지를 결정해주는 접근성(accessibility) 개념입니다.

JavaScript 변수는 3가지 유형의 scope를 가지고 있습니다.

스코프 유형



항목	설명
전역 스코프(Global Scope)	프로그램 전체에서 접근할 수 있는 영역
지역(함수) 스코프 (Local/Function Scope)	함수 내부에서만 접근 가능한 영역
블록 스코프(Block Scope)	{ }(중괄호) 내부에서만 접근 가능한 영역 (let, const 적용)
모듈 스코프(Module Scope)	import, export를 사용하여 파일 단위로 접근 제한

☰ React를 위한 자바 스크립트

☰ `Block Scope

Block Scope는 중괄호 `{ }` 내에서 선언된 변수는 block 외부에서 참조가 불가능함을 의미합니다.

ES6(2015) 이전의 JavaScript 변수는 Global Scope와 Function Scope 2가지를 가지고 있었습니다.

ES6에서 `let`과 `const` 키워드가 추가되면서 Block Scope라는 개념이 추가되었습니다.

사용 예시

```
{  
  let x1 = 10;  
  var x2 = 50;  
}  
  
// 위의 예시에서 변수 x1은 Block Scope를 가지고 있습니다.  
// 따라서, { } 기호를 벗어 나게 되면 접근이 불가능합니다.  
// 반면 변수 x2는 Global Scope로써 { } 기호를 벗어 나더라도 접근이 가능합니다.
```

☰ React를 위한 자바 스크립트

☰ `Function Scope

Function Scope는 함수내에서 선언된 변수입니다.

모든 함수들은 각각의 고유한 scope를 가지고 있습니다.

함수 내에서 var, let 그리고 const로 선언된 변수들은 Function Scope라고 하며, 함수 외부에서 접근이 불가능합니다.

사용 예시

```
function myFunction() {  
  var carName = "Volvo"; // Function Scope  
  let carNo = 100; // Function Scope  
  const maker = "현대"; // Function Scope  
}
```


☰ React를 위한 자바 스크립트

☰ `Global Scope

Global 변수들은 JavaScript program 내의 모든 곳에서 접근이 가능합니다.

함수의 외부에서 선언된 변수들은 Global Scope를 가집니다

var, let, const 키워드를 사용하지 않는 변수들은 자동으로 GLOBAL Scope가 됩니다.

사용 예시

```
var x1 = 20; // Global scope
let x2 = 30; // Global scope
const x3 = 40; // Global scope

myFunction();

// code here can use carName
console.log(carName) ;

function myFunction() {
  carName = "Volvo"; // 전역 변수입니다.
}
```

☰ React를 위한 자바 스크립트

☰ `Template literal

기존의 복잡한 문자열 결합 연산자(+) 기능을 개선한 것이 템플릿 문자열(template literal)입니다.

템플릿 문자열의 특징

외 따옴표 대신 백틱(`)으로 문자열을 사용하여 데이터를 표현합니다.(백틱은 esc 키 아래에 있습니다.)

문자열 내의 변수나 표현식을 해석해 주는 데, 이것을 'string interpolation'이라고 합니다.

`${}` 기호는 템플릿 리터럴 안에서 자바스크립트 표현식을 실행할 수 있게 해주는 문법입니다.

`${변수}` `${수식}` 사용

```
greeting = `${string1} ${string2}`;  
message = `제품 : ${product.name}, 단가 : ${product.price}`;  
operator1 = `연산 결과 : ${value1 * value2}`;
```

유사한 개념

템플릿 문자열은 다음 개념들과 매우 비슷한 개념입니다.

JSTL의 EL `${}` 문법

thymeleaf의 `${}` 문법

☰ React를 위한 자바 스크립트

☰ `Template literal

기존의 복잡한 문자열 결합 연산자(+) 기능을 개선한 것이 템플릿 문자열(template literal)입니다.

`${ }` 기호와 `{ }` 기호

구분	<code>\${ }</code> 기호	<code>{ }</code> 기호
쓰는 곳	백틱(`) 문자열 안	JSX 코드 안
역할	문자열 안에 변수/식 삽입	JSX 안에서 JS 표현식 실행
React 전용?	자바스크립트 기본 문법)	JSX 문법

동시 사용 예시

```
const name = "홍길동";

function Hello() {
  return <h1>{안녕하세요, ${name}님!}</h1>;
}
```

☰ React를 위한 자바 스크립트

☰ `dangerouslySetInnerHTML` 속성

React 전용의 속성(prop)으로 HTML 요소의 innerHTML을 직접 설정할 수 있도록 해주는 특수 속성입니다.

React JSX에서는 문자열을 HTML 태그로 직접 해석하지 않고 일반 텍스트로 처리해 줍니다.

예를 들어서 다음 코드는

```
const html = "<b>굵게</b>";  
<div>{html}</div>;
```

출력 결과가 "굵게"로 보입니다. 즉, 태그 자체를 글자로 인식해서 보여줍니다.

이럴 때, 문자열을 실제 HTML로 렌더링해야 하는 경우 dangerouslySetInnerHTML 속성을 사용하면 됩니다.

사용 예시

```
const result = "<h1 style='color:red'>안녕하세요</h1>";  
  
<div dangerouslySetInnerHTML={{ __html: result }} />
```

React를 위한 자바 스크립트

함수 만들기

자바스크립트의 함수는 `function` keyword와 함수의 이름(name) 및 소괄호() 등으로 구성되어 있습니다.

자바 스크립트 함수 특징

JavaScript에서 함수는 다음과 같은 특징을 가지고 있습니다.

parameter라고 불리는 매개 변수는 콤마를 이용하여 구분 지으면 됩니다.
실행하고자 하는 내용은 중괄호 {} 내에 작성하면 됩니다.

자바 스크립트 함수 유형

항목	설명	사용 예시
함수 선언문 (Function Declaration)	<code>function</code> 키워드를 사용하여 함수를 선언하는 방법입니다. 함수 이름을 지정하고, 괄호 안에 인수를 정의하고, 중괄호 안에 함수의 코드를 작성합니다.	<pre>function functionName(param1, param2, ...) { // 함수의 코드 return result; }</pre>
함수 표현식 (Function Expression)	함수 표현식은 함수를 변수에 할당하는 방법입니다. <code>function</code> 키워드로 함수를 생성한 다음 임의의 변수 이름에 할당하는 방식입니다.	<pre>const functionName = function(param1, param2, ...) { // 함수의 코드 return result; }</pre>
화살표 함수 (Arrow Function)	화살표 함수는 함수 표현식을 더 간단하게 작성할 수 있는 방법입니다. <code>function</code> 키워드를 사용하는 대신 <code>=></code> 기호를 사용합니다.	<pre>const arrowFunctionName = (param1, param2, ...) => { // 함수의 코드 return result; }</pre>

☰ React를 위한 자바 스크립트

☰ `함수 선언문과 `함수 표현식

함수 선언문(Function Declaration)과 함수 표현식(Function Expression)의 장단점을 살펴 보겠습니다.

함수 선언문과 함수 표현식은 다음과 같은 특징들을 가지고 있습니다.

각각의 특징점들을 잘 이해하고 적절히 잘 사용하도록 합니다.

함수 선언문과 표현식 비교

항목	함수 선언문	함수 표현식	비고
호이스팅(Hoisting)	됨	안됨	함수 선언 이전에 미리 호출시 오류 발생 여부
가독성	good	bad	
함수 이름이 필요한가요?	필요함	필요 없음	
미리 할당 가능	X	O	변수나 함수에 할당해주는 기능
고급 기능	X	O	클로저나 콜백 함수 등 사용하기
일반적 사용처	전역 Scope	지역 Scope	

☰ React를 위한 자바 스크립트

☰ `화살표 함수

화살표 함수(Arrow Function)란 function 키워드를 사용하지 않고, 익명으로 함수를 선언하는 방법입니다.

이전 사용 방식

```
hello = function() { /* 함수 표현식 */  
  return "Hello World!";  
}
```

화살표 함수

```
hello = () => {  
  return "Hello World!";  
}
```

화살표 함수

ES6에서 처음으로 소개되었으며, 화살표 함수는 함수 표현식을 간결하게 해주는 장점이 있습니다.

매개 변수와 본문의 블록 사이에 `=>` 기호를 사용합니다.

타 프로그램에서는 이를 람다(lambda) 함수라고 합니다.

☰ React를 위한 자바 스크립트

☰ 화살표 함수

화살표 함수란 `function` 키워드를 사용하지 않고, 익명으로 함수를 선언하는 방법입니다.

기본 구문 형식

기호 `=>`의 왼쪽은 함수로 입력되는 매개 변수를 의미합니다.

항목	설명
<code>() => {statements}</code>	매개 변수가 없는 경우로써 기호 좌측에는 소괄호를 반드시 작성해 주어야 합니다.
<code>(param) => {statements}</code>	매개 변수가 하나인 경우이고, 이와 같은 경우에 소괄호는 optional(선택 사항)입니다.
<code>(param1, param2, ..., paramN) => {statements}</code>	매개 변수가 여러 개인 경우이고, 이와 같은 경우에 소괄호를 반드시 작성해 주어야 합니다.

☰ React를 위한 자바 스크립트

☰ `전개 연산자

나열형 자료(배열, 객체 등)를 개별로 추출하거나 연결하고자 할 때 사용하는 연산자입니다.

코드의 가독성을 높이고 효율적으로 데이터를 처리하기 위해 자주 사용됩니다.

전개 연산자 (...)는 배열이나 객체의 전체 혹은 일부분을 빠르게 복사하기 위하여 사용하는 연산자입니다.

사용 예시

```
const numbers = [1, 2, 3, 4, 5, 6];  
  
const [one, two, ...rest] = numbers;  
// one에 숫자 1이, two에 숫자 2의 값이 저장됩니다.  
// 변수 rest에 나머지 항목 [3, 4, 5, 6]이 저장됩니다.
```

사용상의 장점

코드 간결성 : props와 state 관리가 깔끔해짐.
유연성 : 객체와 배열을 쉽게 병합하거나 수정 가능.
가독성 : 데이터 구조를 직관적으로 다룰 수 있음.

[다음 페이지에 계속...](#)

React를 위한 자바 스크립트

전개 연산자

전개 연산자는 다음과 같은 경우에 사용하면 좋습니다.

항목	설명	예시
props 전달	컴포넌트에 props를 전달할 때, 객체를 전개 연산자로 쉽게 전달할 수 있습니다. 예시에서 `name`과 `age`를 개별적으로 전달하지 않아도 됩니다. 그리고 새로운 props가 추가되어도 코드 변경 없이 전달이 가능합니다.	<pre>const childProps = {name: 'John', age: 25}; <ChildComponent {...childProps} /></pre>
state 업데이트	기존 state 값을 그대로 유지하면서 일부 값만 업데이트할 때 사용됩니다. 예시에서 `...prevState`로 기존 상태를 그대로 유지시키고 `name`의 값만 변경이 됩니다.	<pre>const [state, setState] = useState({name: 'John', age: 25}); // 이름만 업데이트 setState(prevState => ({...prevState, name: 'Jane'}));</pre>
배열 병합 및 추가	배열을 병합하거나 새 요소를 추가할 때 사용됩니다.	<pre>const numbers = [1, 2, 3]; const newNumbers = [...numbers, 4, 5]; console.log(newNumbers); // [1, 2, 3, 4, 5]</pre>
객체 병합	객체를 병합하거나 새로운 속성을 추가할 때 사용됩니다.	<pre>const obj1 = { name: 'John' }; const obj2 = { age: 25 }; const merged = { ...obj1, ...obj2, city: 'New York' }; console.log(merged); // { name: 'John', age: 25, city: 'New York' }</pre>
조건부 props 추가	props를 동적으로 추가하거나 제거할 때 전개 연산자를 활용합니다.	<pre>const isEnabled = true; const buttonProps = { type: 'button', ...(isEnabled && { disabled: false }), }; console.log(buttonProps); // { type: 'button', disabled: false }</pre>

☰ React를 위한 자바 스크립트

☰ 템플릿 문자열

템플릿 문자열에 대한 추가 작업이 필요합니다.

☰ React를 위한 자바 스크립트

☰ 배열 관련 함수

자바 스크립트 배열에 사용 가능한 함수들에 대하여 살펴 봅니다.

배열 관련 함수

항목	설명
sort()	배열 요소들을 알파벳 순서대로 정렬해 줍니다.
reverse()	배열 요소들을 역순으로 정렬해주는 메소드입니다.
concat()	2개 이상의 배열을 합쳐주는 역할을 합니다.
push()	현재 배열 요소의 끝에 새로운 요소를 추가해주는 메소드입니다.
filter()	특정 함수의 조건을 만족하는 요소들을 이용하여 새로운 배열을 만들어 줍니다.
map()	모든 요소들에 대하여 특정 함수를 호출하여 새로운 배열을 만들어 줍니다.
find()	특정 조건에 맞는 요소 중에서 1번째 요소를 찾아 줍니다.
includes()	배열 요소에 항목이 들어 있으면 true 값을 반환합니다.

☰ React를 위한 자바 스크립트

☰ 배열 함수 다루기

sort()/reverse() 등의 배열 관련 메소드에 대하여 다루어 봅니다.

sort() 함수

배열 요소들을 정렬하여, 원본 배열을 변경시킵니다.

기본 값으로 문자열 형태로 인식하여 오름차순으로 정렬이 됩니다.

가장 앞자리의 숫자 "2"가 "1"보다 크므로, "25"는 "100"보다 크다고 인식합니다.

이러한 문제는 Compare Function을 사용하면 해결 가능합니다.

```
array.sort([compareFunction])
```

항목	설명
compareFunction	정렬 순서를 정의해주는 함수입니다. 2개의 매개 변수에 따라서, 1, 0, -1 값을 리턴합니다. function(a, b){return a-b}

React를 위한 자바 스크립트

배열 요소 반복

배열 요소들에 대한 반복 구문을 사용하기 위한 함수입니다.

forEach 구문 특징

제어 변수 `i`와 배열의 크기를 따로 변수로 만들 필요가 없습니다.

사용 예시

다음 예시는 배열 요소들의 값을 10배 곱하기를 수행하는 예시입니다.

```
const numarray = [10, 20, 30];

// item은 1개 요소의 현재 값, index는 해당 요소의 색인 번호,
// arr은 현재 요소 값이 들어 있는 배열 이름
function multiple(item, index, arr) {
    console.log(arr);
    arr[index] = item * 10;
}

numarray.forEach(multiple);
```

`array.forEach(function(currentValue, index, arr), thisValue)`

항목	설명
function	콜백 함수를 의미합니다.
currentValue	현재 처리하고 있는 요소
index	현재 처리하고 있는 요소의 index 값
arr	현재 처리하고 있는 요소의 원본 배열
thisValue	A value passed to the function as its this value (기본 값 : undefined)

☰ React를 위한 자바 스크립트

☰ 배열 함수

배열 안의 각 원소를 변환시키고자 할 때 사용되는 함수로써, 새로운 배열이 만들어집니다.

map() 함수

배열 요소들에 대하여 특정 함수를 실행하여 새로운 배열을 생성해 줍니다.

요소가 비어 있는 배열은 함수가 실행되지 않습니다.

원본 배열에 대한 값 변경은 없습니다.

```
array.map(function(currentValue, index, arr), thisArg)
```

항목	설명
function	콜백 함수를 의미합니다.
currentValue	현재 처리하고 있는 요소
index	현재 처리하고 있는 요소의 index 값
arr	현재 처리하고 있는 요소의 원본 배열
thisArg	callback 함수 내부에서 사용할 this 레퍼런스

☰ React를 위한 자바 스크립트

☰ 배열 함수

배열 요소 중에서 특정 조건에 맞는 데이터만 필터링 해줍니다.

filter() 함수

필터링을 수행하는 특정 함수를 이용하여 새로운 배열을 생성해 줍니다.

요소가 비어 있는 배열은 함수가 실행되지 않습니다.

원본 배열에 대한 값 변경은 없습니다.

```
array.filter(function(currentValue, index, arr), thisValue)
```

항목	설명
function	배열 요소들마다 실행되는 콜백 함수를 의미합니다.
currentValue	현재 처리를 하고 있는 배열 요소
index	현재 처리하고 있는 요소의 index 숫자 값
arr	현재 처리하고 있는 요소의 원본 배열
thisValue	A value passed to the function as its this value (기본 값 : undefined)

React를 위한 자바 스크립트

배열 함수

배열 안의 각 원소들을 반복하면서, 값을 누적하거나 새로운 배열들을 다시 만들 수 있습니다.

App_reduce.js

```
const 배열이름 = [somedata];  
배열이름.reduce((acc, current, index, items) => {  
  // write here todolist.  
}, init);
```

인덱스	0	1	2	3
값	10	15	25	30

항목	설명	필수 여부
acc	최종 결과가 저장되는 곳(변수, 배열 등등)	0
current	배열 요소의 각각의 항목 1개를 의미함	0
index	해당 요소의 인덱스 번호(0부터 시작됨)	X
items	배열 요소들의 목록을 출력해 줍니다.	X
init	acc의 초기 값을 지정함	0

ReactJS 라이브러리

React Component

The React logo, consisting of the word "React" in a bold, blue, sans-serif font, centered within a dark blue rectangular background.

Edit src/App.js and save to reload.

[Learn React](#)

React Component

컴포넌트(Component)의 개요

컴포넌트란 작고 독립적인 코드들의 블록을 의미합니다.

다음 예시는 Top, Content, Bottom라는 컴포넌트들을 별개로 구현하고 이것들을 합쳐서 하나의 화면으로 다시 구성한 예시입니다.

React 맛집

Top

어서 오세요~~맛있는 빵과 음료수가 많이 있습니다.

이름	가격	카테고리
프렌치 바게트	1,000 원	빵
크로와상	2,000 원	빵
아메리카노	3,000 원	음료수
카푸치노	2,000 원	음료수

생성 수정 삭제 카테고리 추가

Bottom

카운터에서 주문해 주시고, 즐거운 시간 되시길 바랍니다.

☰ React Component

☰ Component의 종류

Component는 다음과 같이 2가지 종류가 있습니다.

컴포넌트의 유형에는 클래스형 컴포넌트와 함수형 컴포넌트로 구분이 됩니다.

함수형 컴포넌트는 state가 존재하지 않으므로, SFC(Stateless Functional Component)라고 부르며, life cycle API를 사용할 수 없습니다.



2가지의 컴포넌트는 다음과 같은 특징들을 가지고 있습니다.

항목	함수형 컴포넌트	클래스형 컴포넌트
선언하는 방법	편리	불편
생성자(constructor)	존재하지 않음	존재함
메모리 자원	적게 사용	많이 사용
state 사용 가능 여부	사용 불가능	사용 가능
life cycle API 사용 가능 여부	사용 불가능	사용 가능
hooks 기능	사용 가능	사용 불가능
render() 메소드	사용하지 않아도 됨	반드시 사용해야 함

☰ React Component

☰ Component의 종류

컴포넌트가 작성되는 App.js 파일에 다음과 같은 형식으로 작성합니다.

함수형 컴포넌트가 클래스형 컴포넌트에 비하여 코드가 좀 더 간결합니다.

함수형 컴포넌트에는 render() 구문이 존재하지 않고, extends 키워드를 사용하지 않습니다.

함수형 컴포넌트

```
function App(){
  return(
    <div className="App">
      안녕하세요.
    </div>
  );
}

export default App ;
```

클래스형 컴포넌트

```
import React, {Component} from 'react'

class App extends Component{
  render(){
    return(
      <div>
        내용
      </div>
    );
  }
}

export default App ;
```

☰ React Component

☰ 클래스형 컴포넌트를 구현하는 방법

클래스형 컴포넌트를 구현하는 일반적인 절차는 다음과 같습니다.

구현 지침

Component를 상속 받습니다.

render() 함수는 JSX를 반환하는 역할을 하므로 반드시 구현이 되어야 합니다.

‘export default App;’ 구문은 해당 앱(app)을 외부에서 접근할 수 있도록 해주는 명령어 입니다.

‘export default App;’ 구문은 함수형 컴포넌트에서 동일하게 사용 가능합니다.

```
import React, {Component} from 'react'
import './App.css';

class App extends Component{
  render(){
    return (
      <div className="App">어떤 내용</div>
    );
  }
}

export default App;
```

React Component

컴포넌트를 표현하는 'JSX'

자바스크립트와 XML 문서를 조합하여 사용하여 React에서 사용하는 표현식(Expression)입니다.

'JSX(JavaScript Xml)' A syntax Extension to JavaScript.



JSX의 역할



JSX의 특징

Babel은 브라우저가 최신 자바 스크립트 문법을 이해할 수 있도록 컴파일 해주는 Tool입니다.
ECMAScript는 자바 스크립트 문법적인 문서의 역할을 수행합니다.
하나의 파일에 자바 스크립트와 HTML을 동시에 작성하므로 편리합니다.
자바스크립트에서 HTML을 작성하듯이 하기 때문에 가독성이 높고 작성하기 쉽습니다.

☰ React Component

☰ JSX의 장점

JSX를 사용하면 다음과 같은 장점이 있습니다.

동적 데이터 바인딩

다음과 같이 코드를 간결하게 작성할 수 있으므로, 가독성 향상에 좋습니다.

여기서 중괄호 기호 {}는 JSX 내에서 JavaScript 표현식을 삽입하기 위한 문법입니다.

다음 예시는 동적 데이터 바인딩 예시입니다.

폴더	설명
사용함	<code><div>hello~~{message}</div> /* 변수 message를 출력합니다. */</code>
사용함	<code>{ProductList()} /* ProductList 함수를 호출하여 반환된 JSX를 {} 내에 삽입하는 형태입니다. */</code>
사용안함	<code>React.createElement('div', null, `hello~~\${message}`); /* 기호 백틱(`)을 사용합니다. */</code>

보안성

Injection Attack를 방어할 수 있습니다.

Injection Attack이란 문자열이나 숫자 형식 대신 프로그램 코드를 넣어 악의적으로 실행시키는 동작을 의미합니다.

React Component

중괄호 기호 {}

JavaScript 코드와 JSX 사이의 다리 역할을 하며, 동적 데이터를 JSX에 삽입하는 중요한 역할을 합니다.

JavaScript 표현식

중괄호 안에서는 JavaScript 표현식을 사용할 수 있습니다.

예를 들어 조건문, 연산, 함수 호출 등을 사용할 수 있습니다.

```
const isLoggedIn = true;  
return <div>{isLoggedIn ? "Welcome back!" : "Please log in."}</div>; /* 삼항 연산자 사용하기 */
```

속성 값으로 사용

JSX 요소의 속성 값에도 중괄호를 사용하여 JavaScript 표현식을 적용할 수 있습니다.

```
const imgSrc = "logo.png";  
return <img src={imgSrc} alt="Logo" />;
```

스타일 적용

객체 형태로 스타일을 적용할 때도 중괄호를 중첩해서 사용합니다.

```
const style = {color: 'blue', fontSize: '20px'}; // JavaScript를 의미하는 중괄호 기호  
return <h1 style={style}>Hello World</h1>; // JSX 표현식을 의미하는 중괄호 기호
```

React Component

JSX 사용법

JSX는 다음과 같은 형식으로 사용 가능합니다.

하나의 큰 블록을 요소로 넣을 수 있으며, 최상위 요소는 반드시 1개이어야 합니다.

다음 예시에서 최상위 요소는 `<ul>` 입니다.

열린 엘리먼트와 닫힌 엘리먼트는 반드시 한 쌍(pair)이 되어야 하며, 닫힌 엘리먼트는 `/` 기호를 사용해야 합니다.

```
const myAnimal = (  
  <ul>  
    <li>고양이</li>  
    <li>토끼</li>  
    <li>강아지</li>  
  </ul>  
);
```

열린 엘리먼트와 닫힌 엘리먼트 사이에 내용이 없으면 `/` 기호를 사용하여 종료할 수 있습니다.

다음 예시 `myElement01`에서 `<input>` 태그와 `</input>` 태그 사이에 내용이 없습니다.

따라서, `myElement02`와 같은 방식으로 사용 가능합니다.

`myElement01`와 `myElement02`는 동일하게 인식됩니다.

```
const myElement01 = <input type="text"></input>;  
const myElement02 = <input type="text" />;
```

React Component

JSX 사용법

JSX는 다음과 같은 형식으로 사용 가능합니다.

html에서 사용하는 class 속성은 사용이 불가능하고, 대신 className 속성을 사용해야 합니다.

이유는 JSX가 JavaScript의 특수 형태의 구문이기 때문입니다.

```
const myElement = <h1 className="myclass">Hello World</h1>;
```

class



className



중괄호 {}를 사용하여 React 변수 또는 property 또는 유효한 JavaScript 수식들을 작성할 수 있습니다.

다음 예시는 '사과는 배 보다 15배 더 맛있습니다.'라는 결과를 출력합니다.

```
// {x + y}은 html 코드에서 자바 스크립트의 변수를 참조하는 예시입니다.  
// jsp의 표현식과 유사한 개념입니다.  
const x = 5;  
const y = 10 ;  
const myElement = <h1>사과는 배 보다 {x + y}배 더 맛있습니다.</h1>;
```

```
// forName()은 사용자 정의 함수 이름입니다.  
const someone = (  
  <h1>안녕~~{forName(user)}</h1>  
);
```

☰ React Component

☰ `렌더링(rendering)`

컴포넌트의 UI를 생성하고 업데이트하는 과정입니다.

리액트 컴포넌트는 상태(state)나 속성(props)이 변경될 때마다 UI를 다시 렌더링합니다.

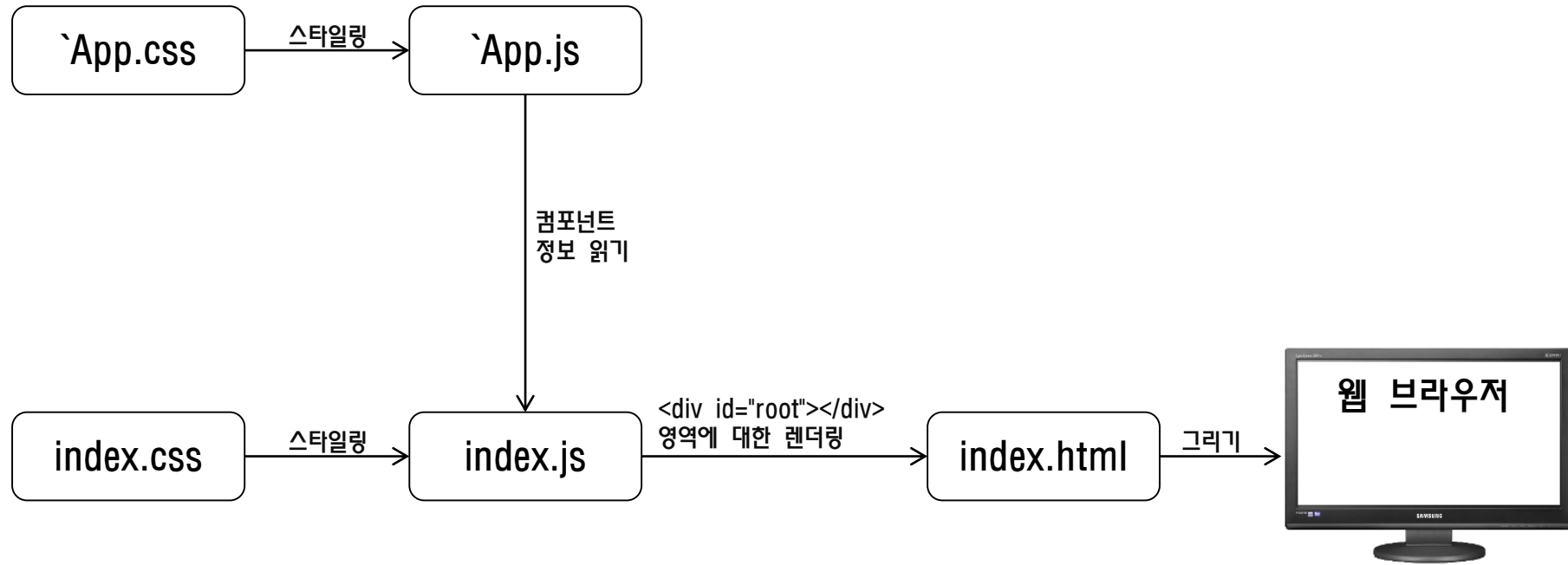
이 과정은 브라우저에 변경 사항을 반영하여 사용자에게 최신 UI를 보여주는 역할을 합니다.

항목	설명
초기 렌더링 (Initial Rendering)	컴포넌트가 처음 화면에 나타날 때의 렌더링입니다. 예를 들어, 사용자가 웹 페이지에 처음 접속하면 리액트가 초기 렌더링을 통해 컴포넌트 트리를 만들어 DOM에 추가합니다.
리렌더링 (Re-rendering)	상태나 속성의 변화로 인해 컴포넌트가 다시 렌더링되는 과정입니다. 리액트는 이러한 변화를 감지하고, 변경된 부분만 업데이트하여 효율적으로 UI를 다시 렌더링합니다.

React Component

리액트의 폴더 구조

create react app을 이용한 설치 후의 폴더 구조는 다음과 같습니다.



☰ React Component

☰ 리액트의 폴더 구조

src 폴더는 사용자가 만들 수 있는 컴포넌트 관련 코딩을 하기 위한 폴더입니다.

폴더	파일	설명
node_modules	-	추가로 설치하는 모듈들이 설치되는 폴더입니다.
public	-	공통으로 사용되는 정적(static) 파일(css, 이미지 등등)을 보관하는 폴더입니다.
public	index.html	react 실행시 결과를 보여 주는 파일입니다. 리액트는 생성된 컴포넌트를 "<div id='root'></div>"라는 영역에 넣어 주도록 미리 약속해 놓았습니다.
src	index.js	엔트리(진입) 파일로써, App.js 파일을 임포트 합니다. document.getElementById('root') 코드가 들어 있습니다.
src	App.js	사용자 정의 컴포넌트를 만들어 주는 파일입니다.
src	index.css	스타일을 지정해주는 파일로써, index.js 파일이 참조하고 있습니다.
src	App.css	스타일을 지정해주는 파일로써, App.js 파일이 참조하고 있습니다.
-	package.json	앱 정보, 전체적인 환경 설정, 스크립트 및 package의 의존성 정보를 저장하고 있습니다.

☰ React Component

☰ 리액트의 주요 파일

`index.js` 파일은 entry point(진입점)로써 앱이 실행되면서 가장 먼저 실행이 됩니다.

React 엘리먼트를 container DOM에 렌더링하고 컴포넌트에 대한 참조를 반환합니다.

소스 코드 예시

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';

# 다음 코드에서 앞쪽의 App은 이 문서에서 사용할 컴포넌트의 이름입니다.
// 그리고, 뒤쪽의 App은 개발자가 작성한 파일의 이름입니다.
import App from './App';

import reportWebVitals from './reportWebVitals';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
```

React Component

리액트의 주요 파일

App.js 파일은 사용자 정의 컴포넌트를 만들어 주는 파일입니다.

일반적인 규약

Component형 컴포넌트는 `return()` 구문을 사용하여 결과물을 반환해 주면 됩니다.
마지막에 코딩된 `export` 구문은 해당 컴포넌트가 외부에서 참조될 수 있도록 지정하는 구문입니다.

소스 코드 예시

```
import React, {Component} from 'react'

class App extends Component{
  render(){
    console.log('초간단 JSX 실습해 보기');
    return (
      <div>
        <div>클래스 방식(초간단 JSX 실습).</div>
        
        <div>반갑습니다.</div>
      </div>
    );
  }
}

export default App;
```

Step. 01 Component를 상속 받습니다.

Step. 2 `render()` 메소드를 구현합니다.

Step. 3 `return` 키워드를 사용하여 내용을 작성합니다.

Step. 4 최상위 요소(가장 바깥 태그)가 반드시 존재해야 합니다.

☰ React Component

☰ 프로퍼티 개요

다음은 중국집에 요리를 시키는 이미지입니다.



중국집이죠?
짜장면 2개랑
짬뽕 3그릇만
배달해 주세요.



아~~넵
배달해
드리겠습니다.



React Component

프로퍼티 개요

컴포넌트 사이에 데이터를 넘기는 예를 살펴 보겠습니다.

다음 예시는 상위 컴포넌트 <App>가 하위 컴포넌트 <Top>에게 'greeting'과 'welcome'이라는 데이터를 넘겨 주고 있는 예시입니다.

하위 컴포넌트 <Top>은 {this.props.greeting}와 {this.props.welcome}라는 구문으로 데이터에 접근이 가능합니다.

```
class Top extends Component{
  render(){
    return(
      <header>
        <h1>{this.props.greeting}</h1>
        <p>{this.props.welcome}</p>
      </header>
    );
  }
}
```

(下位)

this.props

this.props는 해당 컴포넌트에 전달된 속성(props) 목록을 의미합니다.
props는 부모 컴포넌트로부터 전달이 되는 데이터의 목록입니다.

```
class App extends Component{
  render(){
    return(
      <div className="App">
        <Top greeting="인사말" welcome="어서 오세요. 저희 빵집을 방문해 주셔서 감사합니다."/>
        <Content bread01="프렌치 바게트" bread02="크로와상" bread03="브리오슈" bread04="치아바타"/>
        <Bottom goodbye="맏음말" comment="다음 번에도 저희 가게를 방문해 주시길 바랍니다."/>
      </div>
    );
  }
}
```

(上位)

☰ React Component

☰ 프로퍼티 개요

컴포넌트 사이에 데이터를 넘기는 예를 살펴 보겠습니다.

이전 예시에서 상위 컴포넌트가 하위 컴포넌트에게 데이터를 넘겨 주었습니다.

React에서는 이것을 프로퍼티들의 모음이라는 의미로 `properties`라고 부릅니다.

간략히 부르는 용어로 `프롭스(props)`라고 많이 사용합니다.

하위 컴포넌트는 넘겨 받은 `props`를 이용하여 분기(`if`) 혹은 반복문(`for`) 구문에 사용할 수 있습니다.

다만 읽기 전용이므로, 값을 변경하는 것은 불가능합니다.

props의 특징

컴포넌트 내부의 `Immutable Data`(변하지 않는 데이터)입니다.(`read only value`)

상위 컴포넌트에서, 태그 기호(`<>`) 내에 `propsName="value"`의 형식으로 전송합니다.

하위 컴포넌트의 `JSX` 내부에서 중괄호 기호를 사용하여 표현할 수 있습니다.

- `<Button>{props.mytitle}</Button>`

javascript 함수의 매개 변수, html의 속성(`attribute`)을 `JSX`에서는 `props`라고 부릅니다.

HTML	JSX	JavaScript
attribute	props	parameter

☰ React Component

☰ 프로퍼티를 사용한 구현 절차

프로퍼티를 사용한 일반적인 구현 절차는 다음과 같습니다.

하위 컴포넌트

Step. 01(필수) this.props에 대한 프로퍼티를 정의합니다.

Step. 2 특수 변수 propTypes에 해당 프로퍼티의 타입을 지정합니다.

Step. 3 특수 변수 defaultProps에 해당 프로퍼티의 기본 값을 지정합니다.

Step. 4 (필수) return() 함수에서 {} 기호를 사용하여 데이터를 표현합니다.

상위 컴포넌트

Step. 01(필수) 태그 내에 프로퍼티이름 = {someValue} 형식으로 작성합니다.

React Component

생성자(constructor)

생성자는 컴포넌트가 생성이 될 때 필요한 값을 정의하거나, 초기화 하는 역할을 수행합니다.

생성자는 클래스형 컴포넌트에만 존재하는 개념으로, 주로 컴포넌트의 초기 상태를 설정하거나 메소드를 바인딩하는 등의 초기화를 수행합니다.

생성자는 상위 컴포넌트에서 넘어 오는 props를 처리하거나, state를 처리하는 용도로 사용될 수 있습니다.

생성자의 목적

항목	설명
초기 상태 설정 (this.state)	생성자에서 this.state를 사용하여 컴포넌트의 초기 상태를 정의합니다. 이를 통해 컴포넌트가 마운트될 때 상태값을 갖도록 합니다.
메서드 바인딩	이벤트 핸들러와 같은 메서드를 사용할 때, this가 해당 메서드 내부에서 올바른 컴포넌트 인스턴스를 가리키도록 생성자에서 메서드를 bind()로 바인딩할 수 있습니다.
props 초기화 (super(props))	생성자에서 super(props)를 호출하는 이유는 부모 클래스인 React.Component의 생성자를 호출하여 props를 초기화하기 위함입니다. 이 작업을 하지 않으면 this.props에 접근할 때 오류가 발생할 수 있습니다.

this 키워드

`this` 키워드는 주로 클래스형 컴포넌트에서 사용되며, 컴포넌트의 인스턴스를 가리킵니다.

사용 예시

```
class App extends Component {
  constructor(props) { /* 생성자 */
    super(props);
    this.state = {someValue: '하하하'},
  }
}
```

주의 사항

생성자는 선택 사항입니다.

상태 초기화나 메서드 바인딩을 할 필요가 없으면 생성자를 생략할 수 있습니다.

생성자는 클래스형 컴포넌트의 마운트 단계에서 한 번만 호출되며, 그 이후에는 다시 호출되지 않습니다.

☰ React Component

☰ 스테이트 개요

게시물 작성, 상품 주문 등 페이지가 변경되었을 때의 상태 정보를 기록하기 위한 객체입니다.

state의 특징

값을 저장 또는 수정할 수 있는 유동적인 데이터를 의미합니다.

초기 값의 설정이 필수인데, 생성자(constructor)에서 `this.state={}`의 형식으로 설정이 가능합니다.

JSX 내부에서 `{this.state.stateName}`의 형식으로 표현이 가능합니다.

렌더링이 된 이후에는 `this.state`를 사용할 수 없습니다.

값을 변경하고자 하는 경우 `this.setState({...})`를 사용하면 됩니다.

state를 사용한 코딩 구현 절차

Step. 01 생성자를 통하여 state 항목들을 초기화합니다.

Step. 02 `render()` 함수 내에서 `{this.state.stateName}`을 사용하면 됩니다.

위에서 설명한 내용은 클래스형 컴포넌트에 대한 설명입니다.

함수형 컴포넌트는 `useState()` 라는 hook 개념을 사용해야 합니다.

ReactJS 라이브러리

이벤트 다루기

The React logo, consisting of the word "React" in a bold, blue, sans-serif font, centered within a dark blue rectangular background.

Edit src/App.js and save to reload.

[Learn React](#)

이벤트 다루기

Event 추가하기

HTML의 DOM 이벤트처럼, React도 사용자의 event에 기초한 어떠한 동작을 수행할 수 있습니다.

주요 특징

React는 click, change, mouseover 등과 같이 HTML과 동일한 이벤트를 가지고 있습니다.
React의 이벤트는 낙타봉 표기법(camelCase syntax)을 준수해야 합니다.

이벤트 추가하기

on + 이벤트이름의 첫 글자를 반드시 대문자로 작성해야 합니다.

- onClick instead of onclick
- onChange

이벤트 핸들러는 중괄호 기호를 이용하여, 주로 화살표 함수를 많이 사용합니다.

- onClick={shoot} instead of onClick="shoot()"

함수 바인딩

컴포넌트에서 이벤트 발생시 특정한 동작을 수행하기 위하여 함수와 연동시키는 작업을 의미합니다.

바인딩은 크게 2가지 방법이 있습니다.

- 생성자에서 바인딩하기
- render 메소드 내에서 바인딩하기

Event Object

이벤트가 발생한 요소와 관련된 정보를 담고 있는 객체를 의미합니다.

주요 프로퍼티

이벤트 객체는 다양한 프로퍼티를 가지고 있으며, 아래는 일반적으로 사용되는 프로퍼티 몇 가지 예시입니다.

항목	설명
event.target	해당 이벤트를 발생시킨 요소를 의미합니다.
event.currentTarget	이벤트 핸들러가 현재 실행되고 있는 요소를 나타냅니다.
event.type	발생한 이벤트의 종류를 나타냅니다 (예: "click", "keydown" 등).
event.preventDefault()	이벤트의 기본 동작을 취소합니다.
event.stopPropagation()	이벤트의 전파를 중단시킵니다.
nativeEvent	실제 브라우저 이벤트 객체
isDefaultPrevented()	기본 동작이 방지되었는지 확인
isPropagationStopped()	이벤트 전파가 중지되었는지 확인

이벤트의 종류에 따라 다른 프로퍼티가 존재할 수 있습니다.

예를 들어, click 이벤트의 경우에는 클릭 위치와 관련된 정보를 담은 event.clientX와 event.clientY 등의 프로퍼티를 사용할 수 있습니다.

이벤트 다루기

Mouse Events

React에서 사용 가능한 이벤트와 각 이벤트가 가지는 주요 속성들을 살펴 보도록 하겠습니다.

이벤트 이름

``onClick``, ``onDoubleClick``, ``onMouseDown``, ``onMouseUp``, ``onMouseMove``, ``onMouseEnter``, ``onMouseLeave``, ``onMouseOver``, ``onMouseOut``

주요 속성

항목	설명
<code>button</code>	눌린 마우스 버튼 (0: 왼쪽, 1: 휠, 2: 오른쪽)
<code>buttons</code>	눌린 모든 버튼의 비트 마스크 값
<code>clientX</code> , <code>clientY</code>	뷰포트 기준 마우스 좌표
<code>pageX</code> , <code>pageY</code>	문서 기준 마우스 좌표
<code>screenX</code> , <code>screenY</code>	화면 기준 마우스 좌표
<code>ctrlKey</code> , <code>shiftKey</code> , <code>altKey</code> , <code>metaKey</code>	조합 키 상태
<code>relatedTarget</code>	마우스 이벤트와 연관된 대상 (예: <code>`onMouseOver`</code> 에서 대상 요소)

이벤트 다루기

Keyboard Events

React에서 사용 가능한 이벤트와 각 이벤트가 가지는 주요 속성들을 살펴 보도록 하겠습니다.

이벤트 이름

``onKeyDown``, ``onKeyPress`` (이벤트가 비표준화됨, 대신 ``onKeyDown`` 사용 권장), ``onKeyUp``

주요 속성

항목	설명
key	눌린 키의 값 (예: "a", "Enter")
code	물리적 키 코드 (예: "KeyA", "Enter")
keyCode	키의 숫자 코드 (비표준화됨, 사용 지양)
ctrlKey, shiftKey, altKey, metaKey	조합 키 상태
repeat	키가 길게 눌렸는지 여부

이벤트 다루기

Form Events

React에서 사용 가능한 이벤트와 각 이벤트가 가지는 주요 속성들을 살펴 보도록 하겠습니다.

이벤트 이름

``onChange``, ``onInput``, ``onSubmit``, ``onInvalid``, ``onFocus``, ``onBlur``, ``onReset``

주요 속성

항목	설명
target	이벤트가 발생한 요소
value	입력 필드의 값 (주로 <code>`onChange`</code> 에서 사용)
checked	체크 박스 상태
relatedTarget	초점 변경 시 연관된 요소 (예: <code>`onBlur`</code> 또는 <code>`onFocus`</code>)

이벤트 다루기

Drag Events

React에서 사용 가능한 이벤트와 각 이벤트가 가지는 주요 속성들을 살펴 보도록 하겠습니다.

이벤트 이름

``onDrag`, `onDragStart`, `onDragEnd`, `onDragEnter`, `onDragLeave`, `onDragOver`, `onDrop``

주요 속성

항목	설명
<code>dataTransfer</code>	드래그 앤 드롭과 관련된 데이터 객체
<code>setData(type, data)</code>	데이터 설정
<code>getData(type)</code>	데이터 가져오기
<code>clientX, clientY</code>	드래그 위치의 좌표

이벤트 다루기

Touch Events

React에서 사용 가능한 이벤트와 각 이벤트가 가지는 주요 속성들을 살펴 보도록 하겠습니다.

이벤트 이름

``onTouchStart`, `onTouchMove`, `onTouchEnd`, `onTouchCancel``

주요 속성

항목	설명
<code>touches</code>	화면에 닿은 모든 터치 포인트
<code>targetTouches</code>	특정 대상에 닿은 터치 포인트
<code>changedTouches</code>	현재 이벤트에서 변경된 터치 포인트
<code>clientX, clientY</code>	터치 좌표 (뷰포트 기준)

이벤트 다루기

Pointer Events

React에서 사용 가능한 이벤트와 각 이벤트가 가지는 주요 속성들을 살펴 보도록 하겠습니다.

이벤트 이름

``onPointerDown`, `onPointerUp`, `onPointerMove`, `onPointerEnter`, `onPointerLeave`, `onPointerOver`, `onPointerOut`,
`onGotPointerCapture`, `onLostPointerCapture``

주요 속성

항목	설명
<code>pointerId</code>	포인터의 고유 ID
<code>pointerType</code>	포인터 유형 (예: "mouse", "pen", "touch")
<code>width, height</code>	포인터의 접촉 영역 크기
<code>pressure</code>	포인터 압력 (0 ~ 1 사이 값)

이벤트 다루기

UI Events

React에서 사용 가능한 이벤트와 각 이벤트가 가지는 주요 속성들을 살펴 보도록 하겠습니다.

이벤트 이름

``onScroll`, `onResize``

주요 속성

항목	설명
detail	이벤트와 관련된 세부 정보 (예: 클릭 횟수)
view	이벤트가 발생한 뷰 객체

이벤트 다루기

기타 이벤트

React에서 사용 가능한 이벤트와 각 이벤트가 가지는 주요 속성들을 살펴 보도록 하겠습니다.

이벤트 이름

- **Clipboard Events**: ``onCopy``, ``onCut``, ``onPaste``
 - 주요 속성: ``clipboardData`` (복사/잘라내기/붙여넣기 데이터 관리)
- **Media Events**: ``onPlay``, ``onPause``, ``onEnded``, ``onVolumeChange``
 - 주요 속성: ``currentTime``, ``duration``, ``volume``
- **Animation Events**: ``onAnimationStart``, ``onAnimationEnd``, ``onAnimationIteration``
 - 주요 속성: ``animationName``, ``elapsedTime``
- **Transition Events**: ``onTransitionEnd``
 - 주요 속성: ``propertyName``, ``elapsedTime``

ReactJS 라이브러리

React Hooks

The React logo, consisting of the word "React" in a bold, blue, sans-serif font, centered within a dark blue rectangular background.

Edit src/App.js and save to reload.

[Learn React](#)

React Hooks

Hooks

state에 접근하여 특정한 동작을 수행하거나 생명 주기(lifecycle) 등의 메소드를 제어하기 위하여 사용합니다.

Hooks의 특징

컴포넌트의 state와 다른 React features에 대한 값을 수정하기 위한 용도로 사용되는 것입니다.
제약 조건으로 함수형(function) component에서만 사용 가능합니다.

Hooks의 사용 규칙

Hook은 함수형 컴포넌트에서만 호출이 가능합니다.
Hooks can only be called at the top level of a component.
Hooks cannot be conditional

Hooks를 사용하면 잇점

클래스형 컴포넌트의 숨어 있는 코드가 직관적이지 못했습니다.
예를 들면, this.props, this.state 등등이 그러했습니다.
클래스형 컴포넌트의 복잡한 상태(state)와 생명 주기(lifecycle) 관리를 위해서는 간결히 하여 코드 재사용성을 높였습니다.
함수형 프로그래밍의 장점인 순수 함수(pure function)와 불변성(Immutability)을 사용할 수 있습니다.

React Hooks

Hooks

state에 접근하여 특정한 동작을 수행하거나 생명 주기(lifecycle) 등의 메소드를 제어하기 위하여 사용합니다.

Hooks의 종류

리액트의 Hooks는 'use'라는 접두사로 시작하는 함수의 일종입니다.

용도	관련 Hooks
컴포넌트 데이터 관리	useMemo, useCallback, useState, useReducer
생명 주기(life cycle)	useEffect, useLayoutEffect

React Hooks

useState Hook

함수형 컴포넌트에서 state 값을 추적할 수 있도록 해주는 Hook 입니다.

통상적으로 state는 애플리케이션에서 값을 추적할 필요가 있는 properties 또는 data를 참조하는 용도로 사용됩니다.

useState Hook을 사용하기 위하여 다음과 같이 import 구문을 사용해야 합니다.

- `import {useState} from "react";`

useState 초기화

useState를 호출하면 두 개의 값을 반환해 줍니다.

state의 현재 값

state를 수정하기 위한 함수(setter) ← for Update

사용 예시

다음은 color라는 state를 정의하고, 기본 값을 빨간 색으로 지정하고 있습니다.

그리고, 이 값을 변경하기 위한 함수 이름은 setColor입니다.

```
import {useState} from "react";

function FavoriteColor() {
  const [color, setColor] = useState("red");
}
```

React Hooks

useState Hook

함수형 컴포넌트에서 state 값을 추적할 수 있도록 해주는 Hook 입니다.

state 값 읽기

`{}` 기호를 사용하여 컴포넌트 임의의 위치에서 state의 값을 읽을 수 있습니다.

```
<h1>My favorite color is {color}!</h1>
```

state 값 수정

값을 수정하기 위하여 `color = "blue"`와 같은 구문은 허용되지 않습니다.

setter라고 하는 update 함수를 사용 해야 합니다.

```
<h1>My favorite color is {color}!</h1>
<button type="button" onClick={() => setColor("blue")}>
  클릭시 blue로 변경
</button>
```

☰ React Hooks

☰ useEffect Hook

컴포넌트에 추가적인 동작들을 사용할 수 있도록 해줍니다.

부가적인 동작에는 컴포넌트에서 데이터를 추출하거나, DOM을 직접 수정하거나, 타이머를 사용하는 것들이 포함될 수 있습니다.

useEffect 매개 변수

useEffect는 두 개의 매개 변수를 가지며, 1번째 매개 변수는 필수 사항이며, 2번째 매개 변수는 optional입니다.

```
useEffect(<function>, <dependency>)
```

항목	설명
function	구현하고자 하는 동작을 명시합니다.
dependency	props 또는 state 값을 명시합니다.

React Hooks

useEffect Hook

2번째 매개 변수(dependency)의 사용 유무에 대하여 좀 더 살펴 보겠습니다.

항목	코드 예시	설명
No dependency passed	<pre>useEffect(() => { //Runs on every render });</pre>	2번째 매개 변수가 없는 경우, 매번 rendering 됩니다.
An empty array	<pre>useEffect(() => { //Runs only on the first render }, []);</pre>	2번째 매개 변수에 빈 배열이 있는 경우, 1번만 rendering 됩니다.
Props or state values	<pre>useEffect(() => { }, [prop, state]);</pre>	기본적으로 1번 rendering 됩니다. 2번째 매개 변수의 값이 변경이 될때 마다 무조건 rendering 됩니다.

React Hooks

fetch API

fetch API에 대하여 간략히 살펴 보겠습니다.

URL을 매개 변수로 받아 HTTP 요청을 보내며 응답을 받을 수 있는 기능을 제공하며 비동기적으로 동작합니다.

Promise를 기반으로 하여 비동기 작업을 처리하기 때문에 `async`/`await` 구문과 함께 사용하면 더욱 직관적인 코드 작성이 가능합니다.

기본 사용법

```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    return response.json(); // 또는 response.text() 등의 메소드를 사용하여 응답 본문을 처리
  })
  .then(data => {
    console.log(data); // 처리된 데이터
  })
  .catch(error => {
    console.error('There has been a problem with your fetch operation:', error);
  });
```

Promise 객체

JavaScript의 Promise는 비동기 작업을 처리하기 위한 객체입니다.

비동기 작업의 성공 또는 실패를 표현하고, 그 작업이 완료된 후 결과 값을 처리할 수 있는 메소드를 제공합니다.

Promise는 콜백 지옥(callback hell)을 피하고, 비동기 코드를 더 읽기 쉽게 만드는 데 도움이 됩니다.

☰ React Hooks

☰ fetch API

기본 값으로 GET 요청을 전송합니다.

POST, PUT, DELETE 등 다른 HTTP 메소드를 사용하려면 두 번째 인수로 옵션 객체를 전달해야 합니다.

기본 사용법

```
fetch('https://api.example.com/data', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    key: 'value'
  })
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error('Error:', error));
```

☰ React Hooks

☰ fetch API

fetch API는 응답 객체를 제공하며, 이 객체에서 사용하는 주요 메소드는 다음과 같은 항목들이 있습니다.

응답 객체

항목	설명
response.json()	JSON 형식으로 응답 본문을 반환합니다.
response.text()	텍스트 형식으로 응답 본문을 반환합니다.
response.blob()	바이너리 데이터 형식으로 응답 본문을 반환합니다.
response.formData()	FormData 객체로 응답 본문을 반환합니다.
response.headers	응답 헤더에 접근할 수 있는 Headers 객체를 반환합니다.

☰ React Hooks

☰ async/await 사용

fetch를 async/await 구문과 함께 사용하면 코드가 더 깔끔하고 읽기 쉬워집니다.

async/await 사용

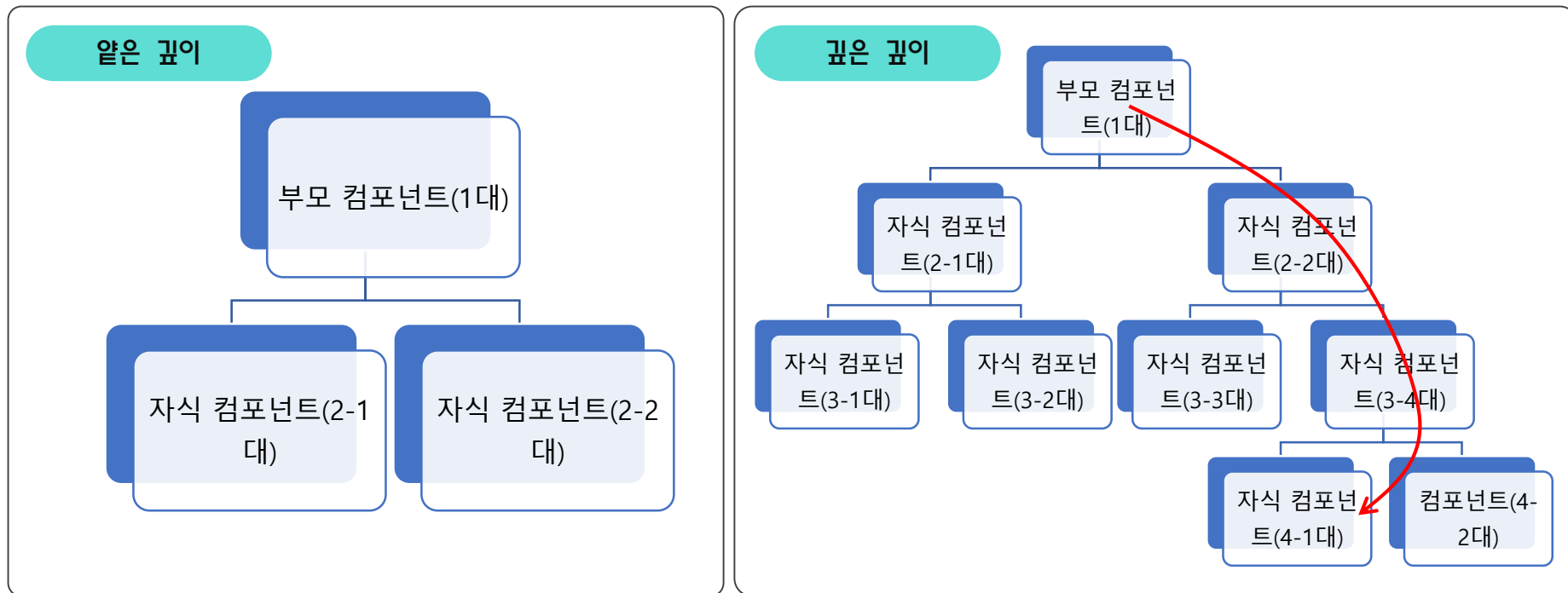
```
async function fetchData() {  
  try {  
    const response = await fetch('https://api.example.com/data');  
    if (!response.ok) {  
      throw new Error('Network response was not ok');  
    }  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error('There has been a problem with your fetch operation:', error);  
  }  
}
```

React Hooks

useContext Hook

state를 전역적(globally)으로 관리하기 위한 방법입니다.

최상위 계층 컴포넌트에 넣어 있는 state를 가장 하단에 있는 컴포넌트에서 접근을 하려면 중간 과정에 있는 컴포넌트에 props 형태로 넘겨 주어서 처리합니다.
"prop drilling"은 연결되어 있는 모든 컴포넌트마다 props를 보내어 처리한다는 용어입니다.
context를 사용하면 이러한 문제를 해결할 수 있습니다.

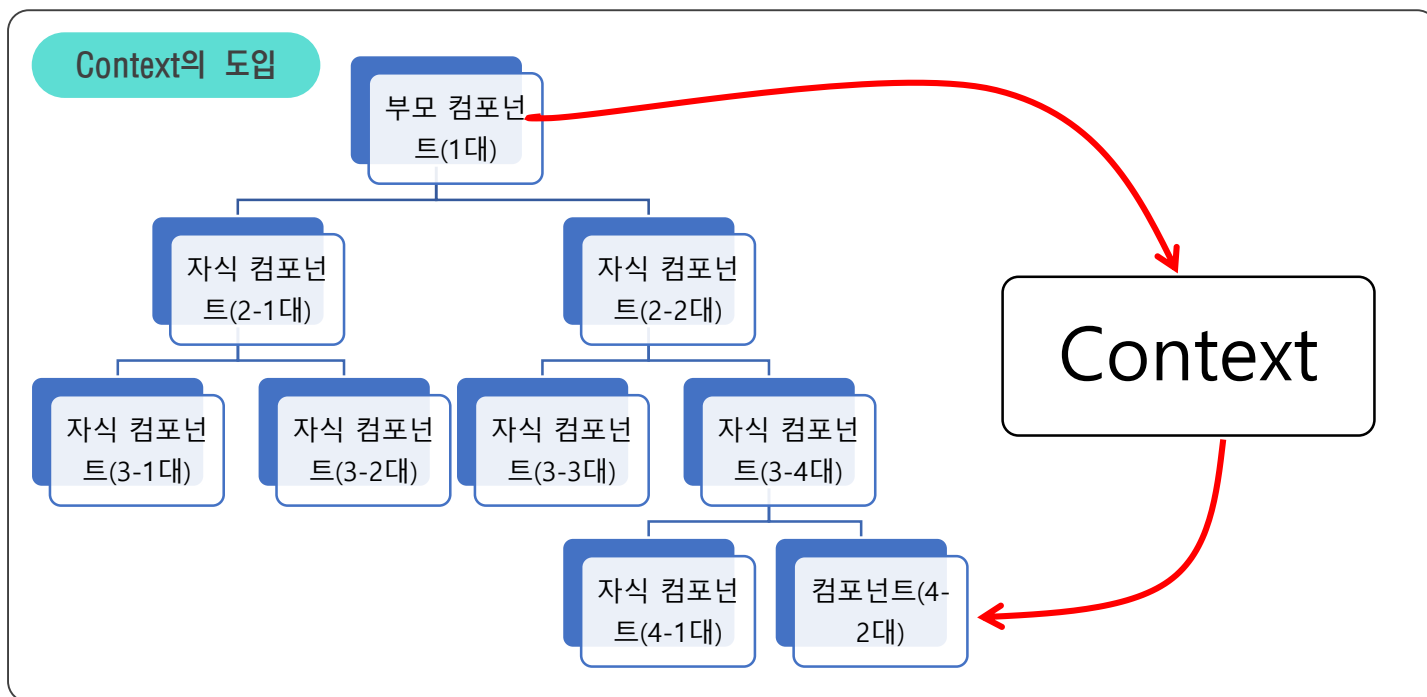


React Hooks

useContext Hook

state를 전역적(globally)으로 관리하기 위한 방법입니다.

Context는 전역적으로 데이터를 공유하기 위하여 사용됩니다. 중간 단계의 Component를 모두 거치지 않고, 사용하고자 하는 Component에서 직접 접근이 가능합니다. React Hooks의 한 유형인 useContext는 이러한 Context를 좀 더 편하게 사용할 수 있게 해주는 역할을 해 줍니다.



☰ React Hooks

☰ useContext Hook

useContext를 사용하는 일반적인 절차에 대하여 우선 살펴 보겠습니다.

useContext 사용하기

createContext를 import 하고, 컨텍스트 객체를 생성합니다.

```
import {useState, createContext} from "react";  
const MyContext = createContext()
```

Context Provider는 하위 컴포넌트를 감싸 주며, state 값을 공급해주는 역할을 합니다.
데이터 공급은 value를 이용하면 됩니다.

```
const [member, setMember] = useState("김철수");  
  
return (  
  <MyContext.Provider value={member}>  
    <h1>{`안녕하세요~~${member}님!!`}</h1>  
    <Child/>  
  </MyContext.Provider>  
);
```

Provider를 사용하였기 때문에, tree 내에 있는 모든 컴포넌트는 MyContext라는 Context에 접근 가능합니다.

☰ React Hooks

☰ useRef Hook

화면이 다시 그려 지더라도 특정한 값을 지속적으로 보존하고자 할때 사용됩니다.

Dom 요소를 직접 접근하거나, 화면이 업데이트 되었을 때 변수에 값을 저장하여 값을 보존할 수 있습니다.

React Hooks

useReducer Hook

useReducer Hook은 useState Hook과 유사하며, 많은 state 값들을 추적하고자 할 때 유용합니다.

useReducer를 사용하면 컴포넌트의 상태 업데이트 로직을 컴포넌트에서 별개로 분리할 수 있습니다.

상태 업데이트 로직을 컴포넌트 바깥에 작성 할 수도 있고, 심지어 다른 파일에 작성 후 불러 와서 사용 할 수도 있습니다.

reducer란 무엇인가요

Reducer(리듀서)는 현재 상태(state) 객체와 행동(action) 객체를 매개 변수로 받아 와서 새로운 상태를 반환해주는 함수입니다.

```
function reducer(state, action) {  
  // some logic  
  return newState;  
}
```

action

업데이트를 하기 위한 어떤 정보를 가지고 있습니다.

주로 객체 형태로 많이 사용되고, 멤버 변수로 type이란 값을 식별자로 많이 사용합니다.

```
// 카운터에 1을 더하는 액션  
{  
  type: 'INCREMENT'  
}
```

React Hooks

useReducer Hook

useReducer Hook은 useState Hook과 유사하며, 많은 state 값들을 추적하고자 할 때 유용합니다.

useReducer 매개 변수

useReducer는 두 개의 매개 변수를 가지고 있습니다.

```
useReducer(reducer, initialState)
```

항목	설명
reducer	state 처리를 위한 로직을 저장하고 있는 리듀서 함수를 의미합니다.
initialState	배열이나 객체 등을 사용한 초기 상태를 저장하고 있습니다.

useReducer Hook을 실행하게 되면 currState와 dispatch 메소드를 반환해 줍니다.

여기서 currState는 현재 state를 의미하며, dispatch는 액션을 발생시켜 주는 메소드입니다.

```
const [currState, dispatch] = useReducer(reducer, initialState);
```

React Hooks

useReducer Hook

useReducer Hook을 정의하는 일반적인 절차입니다.

Step. 01

초기 상태 값들을 설정합니다.

Step. 2

state 처리를 위한 로직을 담고 있는 사용자 정의 reducer를 구현합니다.
이때 액션의 타입과 구현할 동작을 미리 정해 놓으면 빨리 작성할 수 있습니다.

Step. 3

useReducer() 함수를 호출합니다.
현재 state의 정보와 dispatch 메소드가 반환됩니다.

Step. 4

dispatch 함수를 필요한 곳에 작성합니다.

ReactJS 라이브러리

axios, rechart, chart.js

The React logo, consisting of the word "React" in a bold, blue, sans-serif font, centered within a dark blue rectangular background.

Edit src/App.js and save to reload.

[Learn React](#)

☰ axios, rechart, chart.js

☰ axios 개요

HTTP 클라이언트에서 사용하는 라이브러리로, 서버와의 HTTP 요청을 보내고 응답을 받는 데 사용이 됩니다.

주요 특징

Promise API를 기반으로 하는 HTTP Client 프로그램입니다.
비동기 방식으로 HTTP 요청 또는 Rest API 요청을 처리합니다.
성공(then 구문) 또는 실패(catch 구문)에 따른 분기 처리가 가능합니다.
다양한 설정 옵션을 제공합니다.
필요에 따라 요청 헤더, 요청 파라미터 등을 설정할 수 있습니다.
리액트 뿐만 아니라 브라우저, Node.js 등에서도 사용 가능합니다.

주요 기능

브라우저(XMLHttpRequests) 요청 생성
Node.js(http) 요청 생성
Promise API 지원
요청/응답 차단(Intercept)
요청/응답 데이터 변환
JSON 데이터 자동 변환

≡ Promise

자바스크립트에서 비동기 작업의 결과를 나타내는 객체입니다.

일반적으로 시간이 오래 걸리는 작업이나 네트워크 요청과 같은 작업들은 비동기 작업으로 처리해야 합니다.

Promise는 이러한 비동기 작업이 완료되었을 때 결과를 처리하고, 성공 또는 실패에 따른 처리를 할 수 있도록 도와 주는 역할을 합니다.

Promise를 사용하면 비동기 작업을 좀 더 효율적으로 관리하고, 연속적인 비동기 작업을 순차적으로 처리하거나 병렬로 처리할 수 있습니다.

ES6부터는 JavaScript에 내장된 Promise 객체가 제공이 되고 있으며, 많은 라이브러리와 API에서 Promise를 사용하고 있습니다.

Promise의 세 가지 상태

항목	설명
대기(Pending)	비동기 작업이 아직 완료되지 않은 상태입니다.
이행(Fulfilled)	비동기 작업이 성공적으로 완료되어 결과 값이 반환된 상태입니다.
거부(Rejected)	비동기 작업이 실패하거나 오류가 발생한 상태입니다.

Promise의 주요 메소드

항목	설명
then()	Promise가 이행되었을 때 실행할 콜백 함수를 등록합니다. 콜백 함수는 Promise가 이행된 결과를 전달받습니다.
catch()	Promise가 거부되었을 때 실행할 콜백 함수를 등록합니다. 콜백 함수는 Promise가 거부된 이유를 전달받습니다.
finally()	Promise가 이행되거나 거부된 후에 실행할 콜백 함수를 등록합니다. 성공 또는 실패 여부에 상관없이 항상 실행됩니다.

≡ axios, rechart, chart.js

≡ axios 통신

응답 스키마는 데이터를 리턴 받을 때, 활용이 가능한데 다음과 같은 항목들이 있습니다.

axios 설치

--save 옵션은 설치 후, package.json 파일에 해당 모듈 추가 하는 옵션입니다.
npm install axios --save

임포트 구문

import axios from 'axios';

메소드	의미	예시
GET	조회	axios.get('/users/1');
POST	등록	axios.post('/users', {id: 'hong', name: '홍길동'});
PUT	수정	
DELETE	삭제	

async/await 키워드

비동기 통신을 하기 위해서 async 키워드를 사용하면 가능합니다.

해당 메소드(GET/POST 등등) 앞에 await 키워드를 사용합니다.

예외 처리를 하기 위하여 try...catch 구문을 사용합니다.

async/await를 사용하는 방식은 ECMAScript 2017(ES8)에 추가된 새로운 기능입니다.

☰ axios, rechart, chart.js

☰ axios 응답 스키마

응답 스키마는 데이터를 리턴 받을 때, 활용 가능한데 다음과 같은 항목들이 있습니다.

관련 스키마	설치 방법	예시
data	서버가 제공한 응답(데이터)입니다.	text, xml, json 등
status	서버 응답의 HTTP 상태 코드입니다.	200
statusText	서버 응답으로 부터의 HTTP 상태 메시지입니다.	'OK'
headers	서버가 응답 한 헤더는 모든 헤더 이름이 소문자로 제공됩니다.	
config	요청에 대해 `axios`에 설정된 구성(config)입니다.	

≡ axios, rechart, chart.js

≡ axios 데이터 요청

axios를 사용해 데이터를 요청하는 방법은 다음과 같습니다.

Get 방식

```
axios.get('/members', {
  params: {
    name: 'value'
  }
})
.then(function (response) {
  console.log('success');
})
.catch(function (error) {
  console.log('failure');
})
.then(function () {
  console.log('always running');
});
```

Post 방식

```
axios.post('/members', {
  firstName: '길동',
  lastName: '홍'
})
.then(function (response) {
  console.log('success');
})
.catch(function (error) {
  console.log('failure');
});
```

사용 예시

다음과 같이 사용할 수 있습니다.

```
axios.get('/members/10').then(function(response) {
  console.log(response.data);
  console.log(response.status);
  console.log(response.statusText);
  console.log(response.headers);
  console.log(response.config);
})
```

☰ axios, rechart, chart.js

☰ 스프링 부트와 연동하기

스프링 부트와 axios를 이용하여 데이터를 읽어 오는 예제를 만들어 보겠습니다.

부트 스트랩의 ListGroup 그룹 객체를 사용합니다

참조 사이트) <https://react-bootstrap.github.io/docs/components/accordion>

항목	설명
소스 파일	ComboGetSpringBoot.js
관련 폴더	ch08.axios_rechart_chartJs\스프링 관련 파일

구현 순서

스프링 부트를 8989 포트로 구동합니다.

리액트에서 스프링 부트를 3000 포트로 호출합니다.

구현할 내용

탭을 클릭하면 해당 내용에 맞는 데이터를 읽어 와서 출력해 줍니다.

데이터 읽기

스프링 부트에서 데이터를 읽어 옵니다.

예제 01

예제 02

예제 03

예제 04

예제 05

☰ axios, rechart, chart.js

☰ chart.js

HTML을 기반으로 하는 chart를 그려 주는 JavaScript library입니다.

chart.js의 특징

다양한 차트 유형(선 그래프, 막대 그래프, 원 그래프, 등)의 차트를 지원합니다.

Scatter, Line, Bar, Pie, Donut, Bubble, Area, Radar, Mixed Chart 등이 지원합니다.

차트의 크기와 레이아웃을 자동으로 조정할 수 있는 반응형 디자인을 지원합니다.

사용자 상호 작용을 위한 마우스 호버, 클릭, 줌 등의 기능과, 이벤트 처리(클릭하거나 선택하는 등)도 가능합니다.

차트의 색상, 레이블, 축 설정, 도구 팁 등 다양한 옵션을 커스터마이징할 수 있습니다.

chart.js 설치

```
npm install chart.js --save
```

☰ axios, rechart, chart.js

☰ rechart

React 애플리케이션에서 사용할 수 있는 JavaScript 차트 라이브러리입니다.

react-chartjs-2의 특징

React에서 chart.js를 사용하기 위한 라이브러리입니다.

"react-chartjs-2"는 React 애플리케이션에서 chart.js를 쉽게 사용할 수 있도록 도와줍니다.

react-chartjs-2 설치

```
npm install react-chartjs-2 --save
```

☰ axios, rechart, chart.js

☰ chart.js 사이트

chart.js와 관련하여 참조 가능한 사이트입니다.

항목	설명
Chart Js	https://www.chartjs.org/docs/latest/
react-chartjs-2 깃허브	https://github.com/reactchartjs/react-chartjs-2
react-chartjs-2	https://www.npmjs.com/package/react-chartjs-2
React 예제 chart.js 사용하기	https://velog.io/@ceres/React-%EC%97%90%EC%84%9C-chart.js-%EC%82%AC%EC%9A%A9%ED%95%98%EA%B8%B0
좋은 예시들	https://reactchartjs.github.io/react-chartjs-2/#/
리액트를 사용한 차트 예시가 있음	https://github.com/reactchartjs/react-chartjs-2

리액트 차트 라이브러리 | create-react-app | react-chartjs-2) <https://penguinoon.tistory.com/238>

그리고, chart.js 사이트에서 많은 옵션들을 확인 요망) <https://www.chartjs.org/>

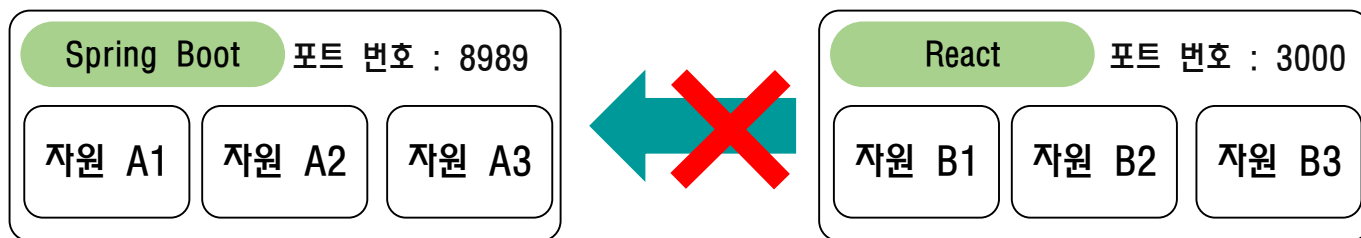
☰ axios, rechart, chart.js

☰ CORS("교차 출처 리소스 공유" 기능)

웹 보안을 유지하면서 다른 출처의 리소스에 접근할 수 있는 메커니즘입니다.

기본 값으로 사이트 A에서 사이트 B의 자원을 읽어올 수 없습니다.

왜냐하면, Origin이 상이(相異)하므로 CORS 정책(policy)에 의하여 블록킹(blocked by) 되기 때문입니다.



SOP(Same-origin policy)

SOP은 동일한 Origin(포트 번호)에만 요청을 보낼 수 있도록 하는 정책을 의미합니다.

다음 구성 요소 중에서 하나라도 동일하지 않으면 SOP 정책에 위반이 되므로, 자원(resource)을 공유할 수 없습니다.

Origin 구성 요소

Origin은 다음과 같은 요소로 구성이 되어 있습니다.

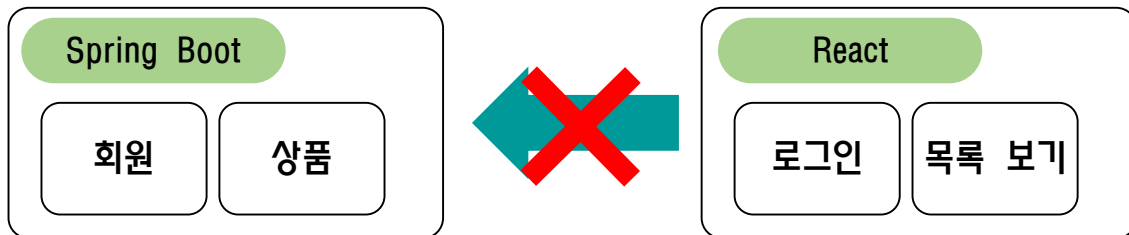
항목	설명
URI Schema	http, https
Hostname	localhost(127.0.0.0), naver.com, daum.net 등등
포트 번호	80, 8989, 3000 등등

☰ axios, rechart, chart.js

☰ CORS policy 오류 문제

React에서 스프링의 자료를 조회하려고 합니다.

CORS 오류 예시



다음은 React(포트 번호 : 3000)에서 Spring Boot(포트 번호 : 8989)로 Json 데이터를 요청했을 때 반환되는 오류 메시지입니다.

이 문제를 해결하려면 우선 CORS 정책에 대한 이해가 필요합니다.

```
localhost/:1 Access to XMLHttpRequest at 'http://localhost:8989/react/data/exam01'
from origin 'http://localhost:3000' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
xhr.js:244
GET http://localhost:8989/thymeleaf/rest/ex01 net::ERR_FAILED 401
```

☰ CORS

"교차 출처 리소스 공유" 기능이라고 하며, Origin이 달라도, 상호 요청을 주고 받을 수 있게 정해 놓은 표준 스펙입니다.

CORS가 필요한 이유

"Cross-Origin Resource Sharing"의 줄인 말입니다.

CORS 설정은 서버 측에서 이루어지며, 서버는 응답 헤더인 "Access-Control-Allow-Origin"을 설정하여 특정 도메인이나 모든 도메인으로부터의 요청을 허용할 수 있습니다.

내용	설명
보안 강화	자신이 서비스하는 동일한 출처의 리소스에만 접근할 수 있도록 제한을 시키고, 다른 외부 출처의 악성 스크립트로부터 보호됩니다.
자원 공유	다른 도메인이나 포트에서 호스팅되는 리소스에 접근하는 경우 CORS를 통해 서로 다른 출처 간에 리소스를 공유할 수 있습니다. 예를 들어, React에서 스프링 애플리케이션의 데이터를 사용하고자 하는 경우, 스프링은 CORS를 허용하여 요청이 제대로 이루어질 수 있도록 해주어야 합니다.
확장성	CORS를 통해 다른 출처의 리소스에 접근하고 상호작용할 수 있으므로, 웹 애플리케이션의 기능과 서비스의 다양성과 확장성을 향상시킬 수 있습니다.

≡ axios, rechart, chart.js

≡ 스프링 부트와 연동하기

스프링 부트에서는 다음과 같이 구현하면 CORS policy 오류 문제를 해결할 수 있습니다.

Spring에서는 @CrossOrigin이라는 어노테이션을 사용하면 간단하게 CORS를 사용할 수 있습니다.

또한, Security를 사용하는 경우에는 별도의 설정 파일에 지정하면 됩니다.

Spring Boot 영역

파일	설명
WebMvcConfig	WebMvcConfigurer를 상속 받아서, addCorsMappings 메소드를 오버라이딩하도록 합니다.

@Configuration

```
public class WebMvcConfig implements WebMvcConfigurer {
```

```
    // 3000번 포트에서 들어온 모든 URL에 대해서 CORS 설정을 적용하겠습니다.
```

```
    @Override // Cors : "교차 출처 리소스 공유"
```

```
    public void addCorsMappings(CorsRegistry registry) {
```

```
        registry.addMapping("/*")
```

```
            .allowedOrigins("http://localhost:3000")
```

```
            .allowedMethods("OPTIONS", "GET", "POST", "PUT", "DELETE");
```

```
    }
```

```
}
```

ReactJS 라이브러리

React Router

The React logo, consisting of the word "React" in a bold, blue, sans-serif font, centered within a dark blue rectangular background.

Edit src/App.js and save to reload.

[Learn React](#)

Router

라우터(Router)

라우터는 URL을 보고 적절한 컴포넌트를 렌더링하는 역할을 합니다.

설치

```
npm install react-router-dom
```

기본 셋팅 방법

index.js 파일에서 앱을 라우터로 감쌉니다.